EHB415 • Dönem Projesi

5G Ağlarına Sensing Entegrasyonu (ISAC)

Temelden uygulamaya öğretici

Bu belge, ekibin tamamının *ne yaptığımızı, neden yaptığımızı ve nasıl yapacağımızı* aynı seviyede anlaması için yazıldı. Soyut bir proposal'dan, çalıştırılabilir ve ölçülmüş bir başlangıca inmeyi hedefliyor.

Bu belgede ne var?

- Sorun ne, biz neyi düzeltmeye çalışıyoruz? (Bölüm 1)
- Temel kavramlar: kaynak ızgarası, scheduler, OFDM radar (Bölüm 2)
- 3GPP standartları: ne söylüyorlar, biz neyi alıyoruz? (Bölüm 3)
- Üç araç: Sionna, ns-3/5G-LENA, OCUDU (Bölüm 4)
- OCUDU kodu: nerede ne var, nereye ekleme yaparız? (Bölüm 5)
- Pilot görevler ve 4 kişilik görev dağılımı (Bölüm 6)
- Makale okuma rehberi ve Google Scholar sorguları (Bölüm 7)

Bu belgedeki sayılar nereden?
Kutular ve şekiller üç türdendir:

- **Ölçülen bulgu**: bu projenin kurulu araçlarında *gerçekten çalıştırılıp* ölçüldü.
- **Pilot görev**: sizin yapacağınız iş.
- **Dikkat**: tuzaklar ve doğrulanmamış yerler.

Sürüm: 2026-10-17 • Araç sürümleri: Sionna 1.2.2, ns-3.48 + 5G-LENA v5.1, OCUDU 26.10.0
Kurulum dokümanları ve betikler: `kurulum/` klasörü • Okuma listesi: `makaleler/OKUMA_LISTESI.md`

İçindekiler

Nasıl okunmalı? 3

1 Büyük resim: sorun ne, biz neyi düzeltiyoruz? 3

1.1 Tek cümlede proje . . . 3
1.2 ISAC nedir? (radar olarak baz istasyonu) . . . 3
1.3 Sorun: sensing bedava değil . . . 4
1.4 Mevcut çalışmada boşluk nerede? . . . 4
1.5 Üç araç, bir hikâye . . . 4
1.6 Araştırma sorusu ve hipotezler . . . 5

2 Temel kavramlar 5

2.1 5G NR radyo kaynak ızgarası . . . 5
2.1.1 Zihninde canlandır: ızgara bir tablodur . . . 6
2.2 gNB'nin iç yapısı: CU, DU, RU . . . 9
2.3 Scheduler: slotta kim ne alır? . . . 10
2.4 OFDM radar: yankıdan menzil ve hız . . . 10
2.5 Sensing sinyali olarak ne kullanılır? . . . 12
2.6 Metrikler: neyi ölçüyoruz? . . . 12

3 3GPP standartları: ne söylüyorlar, biz neyi alıyoruz? 12

3.1 3GPP nedir, bu projede neden önemli? . . . 12
3.2 ISAC'ın standartlardaki yolculuğu . . . 13
3.3 TR 38.765'e rehber (v20.0.0, 2026-06) . . . 13
3.3.1 Değerlendirme senaryosu (Ek A) . . . 13
3.3.2 Sensing kaynak oranı (Bölüm 6.2): bizim ana metriğimiz . . . 14
3.3.3 Ölçüm raporlama seviyeleri (Bölüm 5) . . . 14
3.4 Mimari: SenF ve NsAP . . . 14
3.5 Kanal modeli: hedef, arka plan ve yerel kuplaj . . . 15
3.6 Hangi standart belgelerini bilmeliyiz? . . . 15

4 Üç araç: ne işe yarar, biz ne yaptırırız? 15

4.1 Sionna: fiziksel katman ve ray tracing . . . 15
4.1.1 Haberleşme zamanlaması sensing'i bozar mı? (basit deney) . . . 16
4.2 ns-3 ve 5G-LENA: ağ seviyesi simülasyon . . . 17
4.2.1 Analitik model: sensing kaynak payı ve throughput . . . 18
4.2.2 5G-LENA'ya sensing yükünü nasıl koyarız? (açık tasarım sorusu) . . . 18
4.2.3 Sionna'dan ns-3'e kanal köprüsü: SioLENA . . . 19
4.3 Üç araç ne işe yarar? Tek bakışta . . . 19
4.4 Görsel tur: Sionna ile şehirde ISAC projesi . . . 19

4.5 Görsel tur: OCUDU'da çok kullanıcılı ve dilimli deney . . . 23
4.6 ns-3'ü görmek: NetAnim . . . 24
4.7 OCUDU: gerçek bir gNB yazılımı . . . 24

5 OCUDU kodu: nerede ne var, nereye ekleme yaparız? 25
5.1 Depo haritası . . . 25
5.2 Scheduler bir slotta ne yapar? . . . 26
5.3 Büyük bulgu: OCUDU'da PRS zaten var . . . 26
5.3.1 PRS yapılandırması (YAML) . . . 27
5.3.2 Ölçüm: PRS yükü arttıkça DL throughput ne kadar düşer? . . . 27
5.4 Slicing: PRB oranıyla kaynak ayırma . . . 29
5.5 Nereye ekleme yapabiliriz? . . . 29
5.5.1 Kod geliştirme döngüsü (A ve B için) . . . 29

6 Pilot görevler ve görev dağılımı 30
6.1 Pilot hafta: herkes aynı üç mini deneyi yapar . . . 30
6.1.1 Pilot-1: OCUDU'da PRS maliyetini ölç . . . 30
6.1.2 Pilot-2: ns-3'te UE sayısını artır . . . 31
6.1.3 Pilot-3: Radar ve boşluk deneyi . . . 31
6.2 Dört kişilik görev dağılımı . . . 31
6.3 Hedef grafik: hepimizin üzerinde çalıştığı tek şekil . . . 32
6.4 Haftalık plan önerisi . . . 32
6.5 Riskler ve nasıl yönetiriz . . . 32

7 Makale okuma rehberi 33
7.1 Google Scholar'a ne yazmalı? . . . 33
7.1.1 Scholar'ı verimli kullanmak . . . 33
7.2 Bir makalede ne aranır? (7 soru) . . . 34
7.3 Okuma listesi (indirildi, doğrulandı) . . . 34

A Sözlük 35

B Soru kartlarının beklenen cevapları (yönetici için) 35

C Komut kartı 36

D Ölçüm yöntemi ve sınırlar 36

Nasıl okunmalı?

Herkes **Bölüm 1**'i (büyük resim) okur. Sonra rolüne göre:

Rol	Önce oku	Pilot görevi
Herkes	Bölüm 1, 2 ve 6	Pilot-1, 2, 3 (hepsi)
Standart ve metrik	Bölüm 3	Bölüm 6
Sionna	Bölüm 2.4, 4.1	Pilot-3
ns-3	Bölüm 4.2	Pilot-2
OCUDU	Bölüm 4.3, 5	Pilot-1

Okurken şunu hatırla

Bu proje bir *yazılım yazma* projesi değil, bir *ölçüm ve karşılaştırma* projesi. Hedefimiz yeni bir 5G sistemi yapmak değil; mevcut açık kaynak araçlarla şu soruya kanıtlı bir cevap vermek: **sensing eklemek haberleşmeyi ne kadar etkiler?** Bu yüzden bu belgenin her bölümü o soruya bir şey ekler.

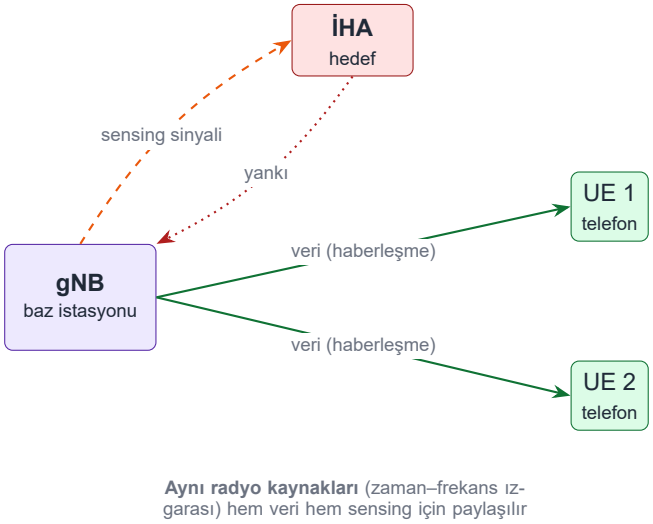
1 Büyük resim: sorun ne, biz neyi düzeltiyoruz?

1.1 Tek cümlede proje

Bir 5G baz istasyonu (gNB) aynı radyo kaynaklarıyla hem **kullanıcıya veri** taşıyıp hem de çevreyi **algılayabilirse** (ISAC: *Integrated Sensing and Communication*), algılama için harcanan kaynak **haberleşme performansından ne kadar götürür**?

1.2 ISAC nedir? (radar olarak baz istasyonu)

Bir radar, sinyal gönderir ve yansımayı dinler. Yansımanın geç gelmesinden **uzaklığı**, frekansının kaymasından (Doppler) **hızı** anlar. Bir 5G baz istasyonu da zaten sürekli sinyal gönderir. ISAC fikri: *bu sinyallerin yansımasını da dinleyip çevreyi izlemek*, örneğin izinsiz bir **insansız hava aracını (İHA)** bulmak.



Şekil 1. ISAC fikri: gNB aynı sinyalle kullanıcılara veri taşır ve İHA'dan dönen yankıyı dinleyerek konumunu ve hızını kestirir.

1.3 Sorun: sensing bedava değil

Radyo kaynakları sonludur. Bir *slot* içindeki bir kaynak ya bir kullanıcıya veri taşır ya da sensing sinyali olarak kullanılır. Sensing'e ne kadar kaynak ayırırsak:

- **Sensing kalitesi artar** (daha az kaçırılan hedef, daha doğru konum),
- **Haberleşme kapasitesi düşer** (daha az throughput, daha çok gecikme).

Bu bir **ödünleşim (trade-off)**. Soru, bu ödünleşimin şeklinin ne olduğu.

1.4 Mevcut çalışmada boşluk nerede?

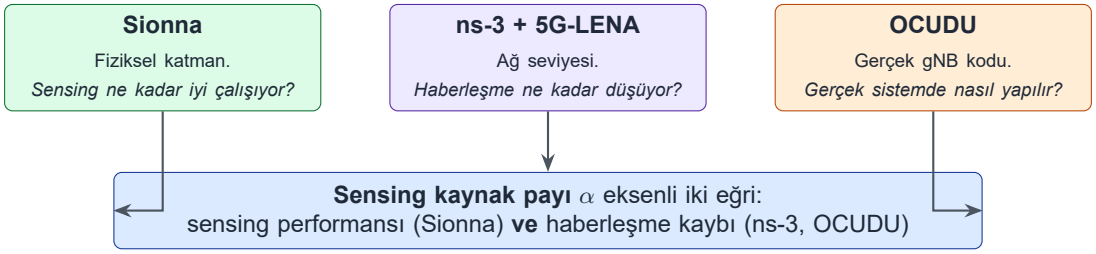
3GPP, ISAC'ı standartlaştırmak için **TR 38.765**'i (Rel-20) yazdı. Bu rapor İHA tespiti için *sensing performansını* değerlendirdi: kaçırma olasılığı, yanlış alarm, konum ve hız doğruluğu. Ama raporun kendisi şunu açıkça yazıyor (*bölüm 6.3.0*): *"The performance of communication is not evaluated."* Yani **haberleşme tarafı değerlendirilmedi**.

Projenin boşluğu, standardın kendi cümlesiyle

3GPP TR 38.765 v20.0.0 bölüm 6.3.0: sensing sonuçları raporlanırken haberleşmenin etkisi *hesaba katılmadı*. Bizim katkımız tam bu eksik taraf: **sensing yükü arttıkça haberleşme ne kadar kaybediyor, ve bunu açık kaynak araçlarla nasıl ölçeriz?**

1.5 Üç araç, bir hikâye

Proposal'daki üç araç bu soruya birbirini tamamlayan üç açıdan bakıyor:



Şekil 2. Projenin yapısı: üç araç, aynı α ekseninde birleşen iki sonuç.

Araç	Cevapladığı soru	Çıktısı
Sionna	Sensing sinyaline ne kadar kaynak verirsek hedefi ne kadar iyi buluruz?	Tespit ve hata eğrileri, range-Doppler haritaları
ns-3 + 5G-LENA	Kaynağın bir kısmı sensing'e gidince birçok kullanıcılı bir ağda throughput ve gecikme ne olur?	Throughput, gecikme, paket kaybı tabloları
OCUDU	Gerçek bir gNB yazılımı bu yükü nasıl zamanlar? Sensing nereye takılır?	Çalışan deney, kod haritası, tasarım önerisi

1.6 Araştırma sorusu ve hipotezler

Ana soru: Sensing için ayrılan radyo kaynağı payı α arttıkça, haberleşme throughput'u ve gecikmesi nasıl değişir; bu, sensing kazancıyla nasıl dengelenir?

H1 Haberleşme throughput'u, sensing'e giden kaynak payı kadar *doğrusal* düşer: $T(\eta) = \min\{\eta,\, 1-\alpha\}$. *(Dickerson vd. [6] modeli; OCUDU ölçümümüzde zaten yaklaşık olarak gördük, bkz. Şekil 26.)*

H2 Sensing sinyali zaman ekseninde *düzensiz boşluklarla* kesildiğinde, sensing kalitesi (özellikle Doppler/hız) hızla bozulur. *(Sionna'da basit bir deneyle gördük, bkz. Şekil 12.)*

H3 Yüksek yükte, kaynakları sensing ve haberleşme arasında *uyarlanır* paylaştıran bir politika, sabit rezervasyondan iyi bir ödünleşim verir. *(Literatürde önerildi [6]; bizim ölçmemiz gereken kısım.)*

Hipotez ne demek?

H1–H3 *kesin sonuç değil*, test edeceğimiz iddialardır. Doğru çıkabilirler de, çıkmayabilirler de. Proje başarısı, hipotezlerin doğru çıkması değil, **iyi ölçülmüş ve açıklanmış sonuç** üretmektir.

2 Temel kavramlar

Bu bölüm, proje boyunca duyacağınız her terimi sıfırdan anlatır. Zorlanırsanız Ek A'teki sözlüğe bakın.

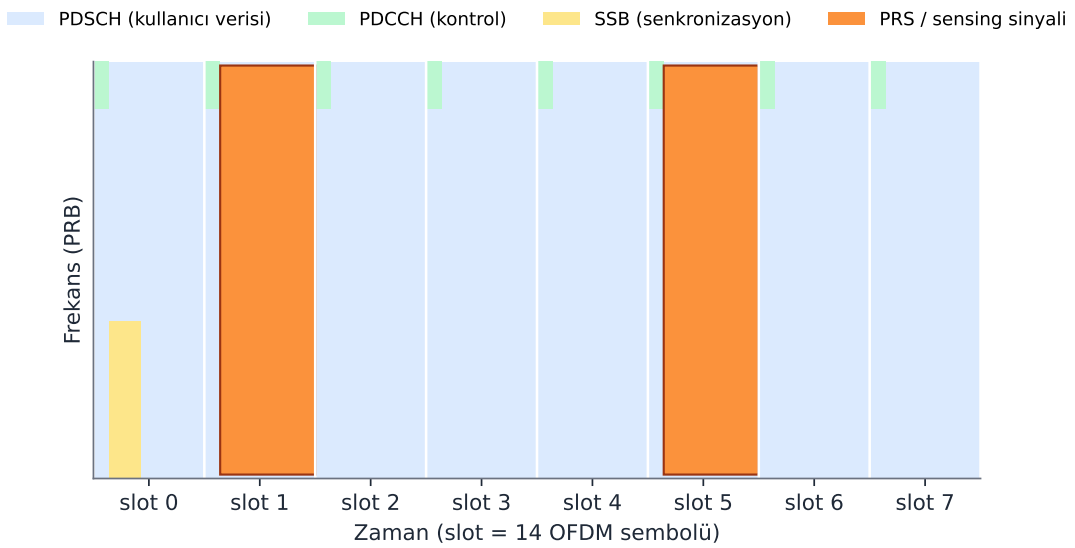
2.1 5G NR radyo kaynak ızgarası

5G NR'de radyo kaynağı bir **zaman–frekans ızgarasıdır**:

- **Zaman ekseni:** bir **slot** 14 **OFDM sembolünden** oluşur. Slot süresi alt taşıyıcı aralığına (SCS) bağlıdır: 15 kHz'de 1 ms, 30 kHz'de 0,5 ms.
- **Frekans ekseni:** 12 alt taşıyıcı bir **PRB**'yi (Physical Resource Block) oluşturur. 20 MHz'lik, 15 kHz'lik bir hücrede yaklaşık 106 PRB vardır.
- En küçük birim **kaynak elemanı (RE)**: 1 sembol $\times$ 1 alt taşıyıcı.

Izgaranın her bölgesinin bir görevi vardır:

Kanal/sinyal	Görevi
PDSCH	Kullanıcı verisi (indirme, *downlink*). Throughput'u belirleyen kaynak.
PDCCH	Kontrol: hangi kullanıcı hangi PRB'yi alıyor? Slotun ilk sembollerinde.
SSB	Senkronizasyon: telefonlar hücreyi bulsun diye periyodik yayın.
PRS	**Konumlandırma referans sinyali** (Rel-16). Bilinen bir dizi içerir, bu yüzden *radar gibi* kullanılabilir.
SRS	Telefonun gönderdiği (yukarı bağlantı) referans sinyali. Kanal ölçümü için.

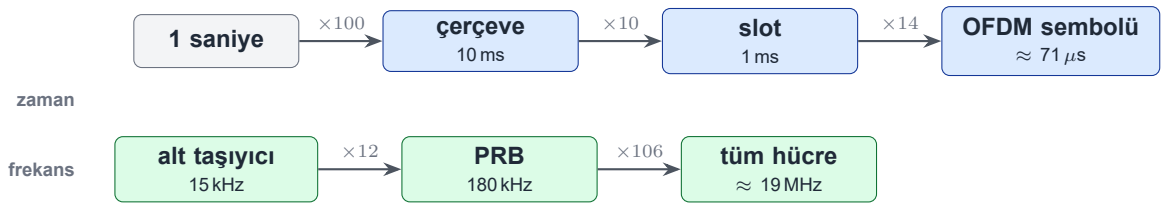


Şekil 3. Şematik kaynak ızgarası (8 slot). Turuncu bloklar slot 1 ve 5'teki sensing/PRS sinyali: kapladığı PRB'ler o slotta kullanıcı verisi için kullanılamaz. Bu şema OCUDU'da PRS periyodu $T = 4$ slot ve ofset 1 olan gerçek bir yapılandırmayı temsil eder.

Sensing bu ızgarada nasıl görünür? Sensing için gönderilen sinyal (örneğin PRS), ızgaranın bir bölgesini kaplar. Orası artık kullanıcı verisi taşıyamaz. Ne kadar bölge kaplandığı, **sensing kaynak payı** α'dır. Bu projenin merkezindeki sayı budur.

2.1.1 Zihninde canlandır: ızgara bir tablodur

Izgarayı **bir Excel tablosu** gibi düşün: *sütunlar zaman* (semboller), *satırlar frekans* (alt taşıyıcılar). Scheduler bu tabloyu her slotta **dikdörtgenlere böler**; her dikdörtgen bir kullanıcının verisidir (ya da bir sinyaldir: SSB, PRS...). Aşağıda önce tablonun ölçeğini, sonra *gerçek bir gNB'nin* bu tabloyu nasıl doldurduğunu göreceksin.



Şekil 4. Zaman ve frekans eksenlerinde "kaç tane ne" (15 kHz alt taşıyıcı aralığı, 20 MHz'lik hücre). Slotun içinde 14 sütun (sembol) ve $106 \times 12 = 1272$ satır (alt taşıyıcı) vardır.

Bir slotun kaç kaynak elemanı var?

$$\underbrace{106}_{\text{PRB}} \times \underbrace{12}_{\text{alt taşıyıcı/PRB}} \times \underbrace{14}_{\text{sembol}} = \mathbf{17\,808} \text{ kaynak elemanı (RE) / slot}$$

Saniyede 1000 slot olduğundan bu, saniyede $\approx 17{,}8$ milyon RE eder. Bir RE 64QAM ile en çok 6 bit taşıyabilir; bu yüzden teorik üst sınır slot başına $17\,808 \times 6 = 106\,848$ bit, yani yaklaşık 107 Mbps'tir. Gerçek sistem bunun altında kalır, çünkü kontrol sinyalleri, referans sinyalleri ve hata düzeltme kodu yer kaplar.

Gerçek bir gNB bu tabloya ne yazıyor? OCUDU'yu çalıştırıp scheduler log'unu açtık (PRS her 4 slotta bir; `ogretici/betikler/ocudu_slot_log.sh`). Log'dan *gerçek* iki satır:

```
[  876.7] PDSCH: rnti=0x44 ... prb=[0, 96) symb=[1, 14) mod=64QAM ... tbs=7941
[  880.1] PUSCH: rnti=0x44 ... prb=[0, 106) symb=[0, 14) mod=64QAM ... tbs=9737
```

Bu tek satırı "ızgara tablosu" diliyle oku:

Log'da	Anlamı	Tabloda nereye karşılık gelir?
`[ 876.7]`	Çerçeve 876, slot 7	876. çerçevenin (10 ms) 8. slotu (15 kHz'de bir çerçevede 10 slot var).
`prb=[0, 96)`	PRB 0'dan 95'e: **96 PRB**	Tablonun *frekans* yönündeki kalınlığı: $96 \times 12 = 1152$ alt taşıyıcı.
`symb=[1, 14)`	Sembol 1'den 13'e: **13 sembol**	Tablonun *zaman* yönündeki uzunluğu. Sembol 0 kontrol kanalına (PDCCH) kalır.
`mod=64QAM`	64QAM modülasyon	Her RE 6 bit taşır (en çok).
`tbs=7941`	Taşıma bloğu: **7941 bayt**	Bu dikdörtgenin taşıdığı veri: $7941 \times 8 = 63\,528$ bit.

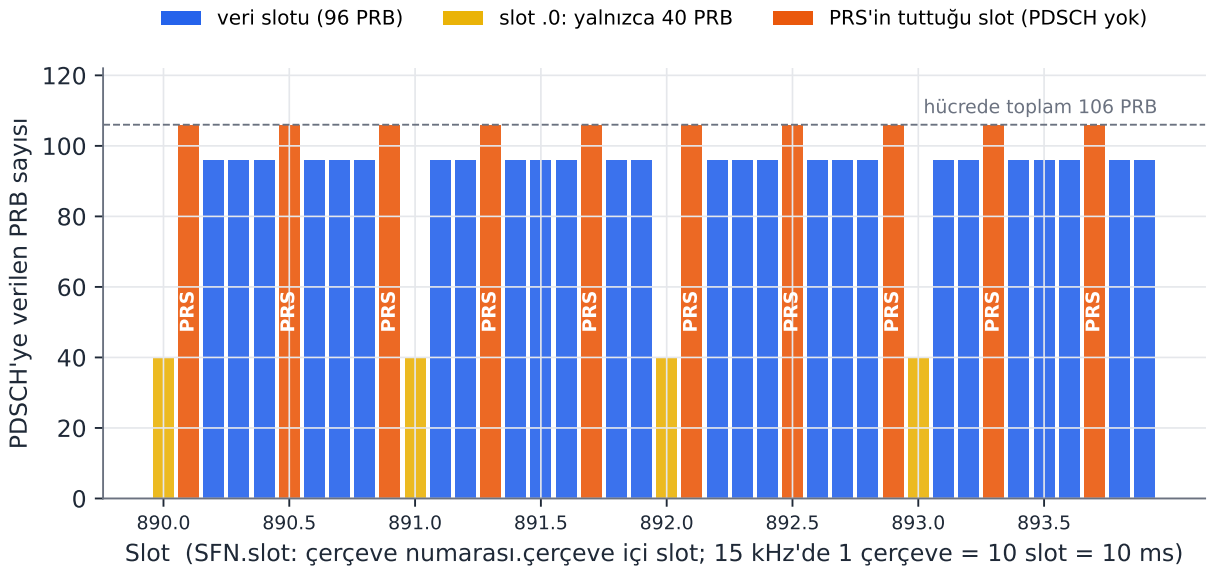
Yani bu kullanıcıya verilen dikdörtgen $96 \times 12 \times 13 = \mathbf{14\,976}$ RE'dir ve $63\,528$ bit taşır: RE başına $63\,528/14\,976 \approx 4{,}24$ bit (6 bitten az, çünkü DMRS referans sinyali ve kodlama yeri tutar). Slot başına $\approx 63{,}5$ kbit ve saniyede 1000 slot $\Rightarrow$ en çok $\approx 63{,}5$ Mbps.

Throughput'u elle hesapladık, ölçümle tuttu

Log'daki 6365 PDSCH'nin ortalama taşıma bloğu 7251 bayt, yani **58,0 kbit/slot** (en çok kullanılan kalıp: 96 PRB, bazen 40 PRB).

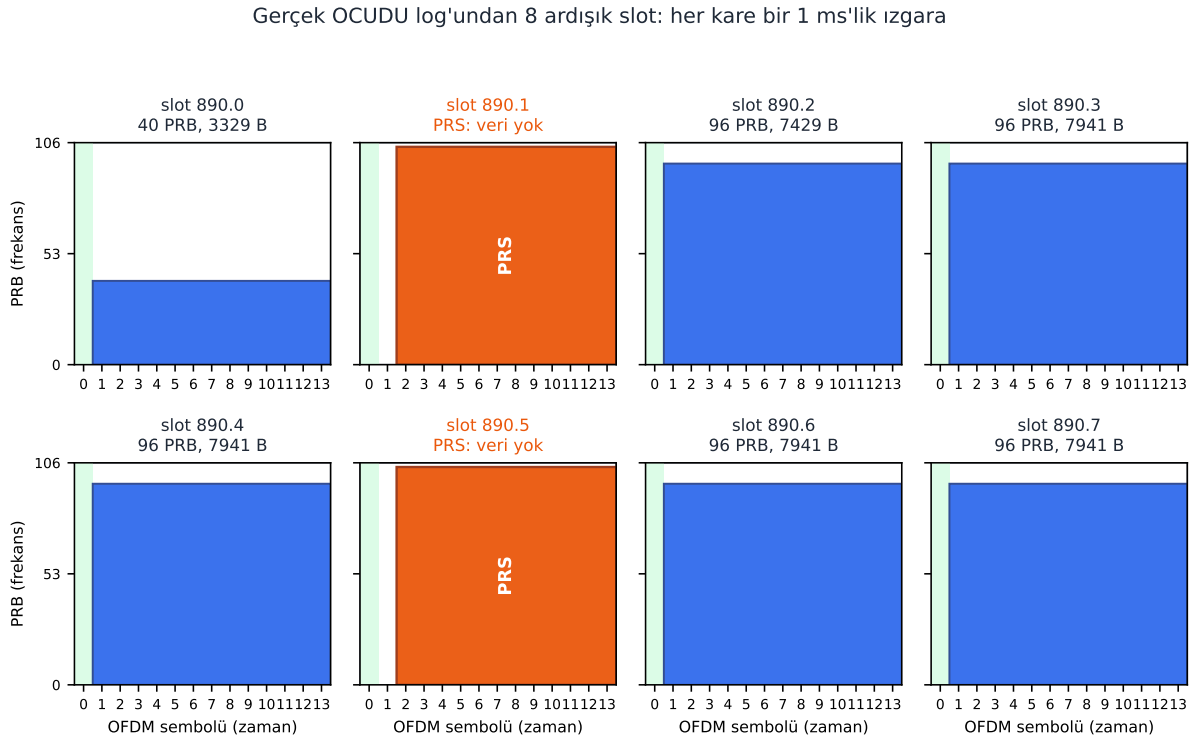
- **PRS yokken:** saniyede 1000 PDSCH'li slot $\Rightarrow 1000 \times 58{,}0 = 58{,}0$ Mbps. Ölçülen: **59,0 Mbps**.
- **PRS her 4 slotta bir iken:** saniyede 750 PDSCH'li slot $\Rightarrow 750 \times 58{,}0 = 43{,}5$ Mbps. Ölçülen: **43,8 Mbps**.

Kural: DL throughput $\approx$ (PDSCH'li slot sayısı / s) $\times$ (taşıma bloğu bit/slot). Sensing bu çarpımın birinci çarpanını (PDSCH'li slot sayısı) ya da ikincisini (PRB sayısı) küçültür. Projenin özü bu iki cümledir.



Şekil 5. Gerçek OCUDU log'undan 40 ardışık slot (SFN 890.0'dan 893.9'a; veri: `ogretici/veri/ocudu_slot_serit.csv`). Her çubuk bir slotu, yüksekliği PDSCH'ye verilen PRB sayısını gösterir. **Turuncu slotlarda PDSCH hiç yok:** bunlar 890.1, 890.5, 890.9, …, yani *tam her 4. slot*. PRS periyodunu 4 slot, ofsetini 1 yapmıştık. Sarı slotlar (`.0`) yalnızca 40 PRB alıyor; muhtemelen aynı slottaki SSB yer tuttuğu için (bunu kodla doğrulamadık).

Bu şekil bölümün en önemli görseli: **PRS'in "kaynağı götürmesi" soyut bir cümle değil, log'da sayılabilir bir şeydir**: her 4. slotta tablonun tamamı PRS'e gider, o slotta kullanıcıya hiçbir şey kalmaz. Ayrıca `symb=[1, 14)` her PDSCH'de aynı: veri hep slotun *tamamına* yayılıyor. Bu, Bölüm 5.3'deki "4 sembollük PRS de 12 sembollük PRS kadar kayıp veriyor" bulgusuyla uyumlu bir açıklama olabilir (PRS'in kapladığı PRB'lere slotun bütününde veri verilemiyor), ama bunu henüz kodla doğrulamadık.



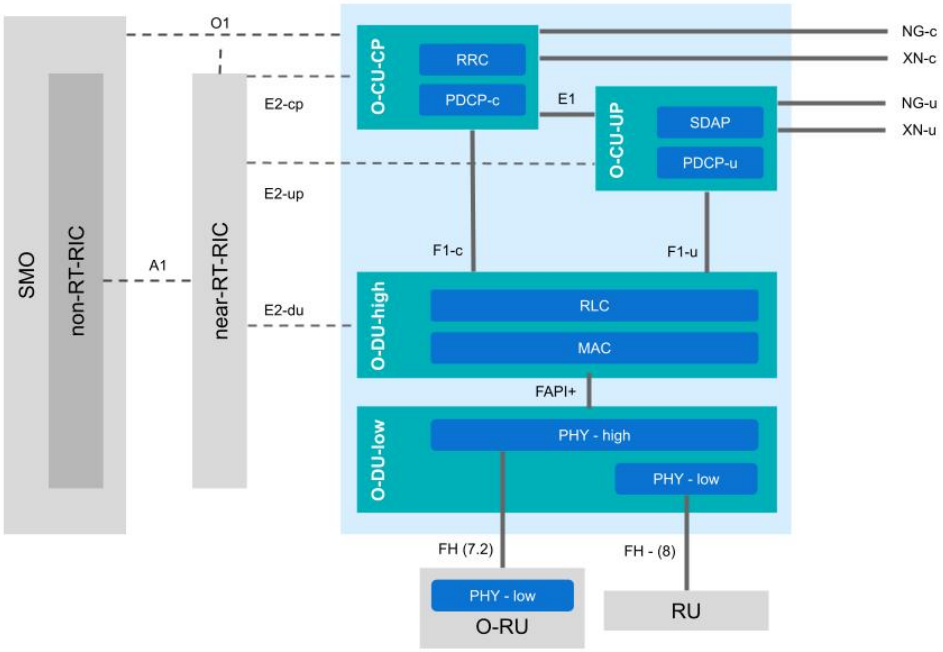
Şekil 6. Aynı veri, bu kez ızgara olarak: 8 ardışık slotun her biri 14 sembol × 106 PRB'lik bir tablo. Mavi dikdörtgen log'daki PDSCH tahsisidir (`prb`, `symb` değerleri log'dan). Turuncu dikdörtgen yapılandırmadaki PRS'tir (periyot 4, ofset 1, 12 sembol, 104 PRB). Soluk yeşil şerit: PDSCH'nin başlamadığı sembol 0 (kontrol alanı). Animasyonlu hâli: `ogretici/sekiller/ocudu_izgara_filmi.gif`.

Kendin dene (20 dakika)

1. **Elle hesapla:** Alt taşıyıcı aralığı 15 yerine **30 kHz**, bant 20 yerine **40 MHz** olsun. Bu durumda PRB sayısı yine 106'dır (3GPP TS 38.101-1'deki tabloya göre; sen de tabloya bakıp doğrula, biz doğrulamadık). (a) Bir slot kaç ms sürer, saniyede kaç slot vardır? (b) Bir slotun RE sayısı değişir mi? (c) Saniyedeki RE sayısı kaç kat artar? (d) Bir PRB artık kaç kHz'dir? *Beklenen cevap:* (a) 0,5 ms, 2000 slot/s. (b) Hayır, 17 808. (c) 2 kat (≈ 35,6 milyon RE/s). (d) $12 \times 30 = 360$ kHz.
2. **Gerçeği gör:** Ubuntu'da `bash ogretici/betikler/ocudu_slot_log.sh` çalıştır. Sonra `grep -a "PDSCH: rnti" /tmp/gnb_slotlog.log | head` ile ilk PDSCH satırlarını bul ve yukarıdaki tabloyla okumayı dene: `prb`, `symb`, `tbs` ne diyor?
3. **PRS slotlarını bul:** `grep -a "Slot decisions" /tmp/gnb_slotlog.log | head -50` satırlarında `0 PDSCHs` geçen slotların numaralarına bak. Aralarındaki fark kaç? (Cevap: 4.)
4. **Değiştir:** `ogretici/veri/ocudu_p4_config.yml` dosyasında `periodicity_slots: 8` yap, tekrar çalıştır. `0 PDSCHs` slotları şimdi kaç slotta bir? Throughput ne olur?

2.2 gNB'nin iç yapısı: CU, DU, RU

Bir gNB tek parça değildir. Açık RAN mimarisinde (O-RAN) işlevler katmanlara bölünür:



Şekil 7. O-RAN gNB mimarisi. Kaynak: OCUDU deposu, `lib/oran_gnb_arch.jpg` (SPDX: BSD-3-Clause-Open-MPI, Software Radio Systems Limited). Biz özellikle **O-DU-high** içindeki **MAC** (scheduler orada) ve **O-DU-low**'daki **PHY** ile ilgileniyoruz.

Parça	Ne yapar?
CU-CP / CU-UP	Merkezi birim: kontrol düzlemi (RRC, bağlantı yönetimi) ve kullanıcı düzlemi (veri yönlendirme, SDAP/PDCP). Çekirdek ağla konuşur.
DU-high	RLC ve **MAC**. MAC'in içindeki **scheduler**, her slotta kaynakları kullanıcılara dağıtır.
DU-low	PHY (fiziksel katman): modülasyon, kodlama, FFT. PRS sinyalini üreten kod burada.
RU	Radyo: antenler ve RF. Bizde gerçek RU yok; yazılımla taklit ediliyor (`ru_dummy`, ZMQ).

2.3 Scheduler: slotta kim ne alır?

Scheduler, *her slotta* (örneğin her 1 ms) bir karar verir: hangi kullanıcı hangi PRB'leri, hangi modülasyonla alacak? Bu kararın sonucu bir "tahsis listesi"dir. Çok kullanıcılı bir ağda throughput'u ve gecikmeyi esas olarak scheduler belirler.

Yaygın politikalar:

- **Round-Robin (RR):** herkese sırayla.
- **Proportional Fair (PF):** kanalı iyi olana öncelik verir ama kimseyi aç bırakmaz. En yaygın politika.
- **QoS-farkında (örn. M-LWDF, EXP/PF):** gecikme bütçesi yaklaşan paketlere öncelik. (Capozzi vd. [17]'nin taraması bu aileyi anlatır.)

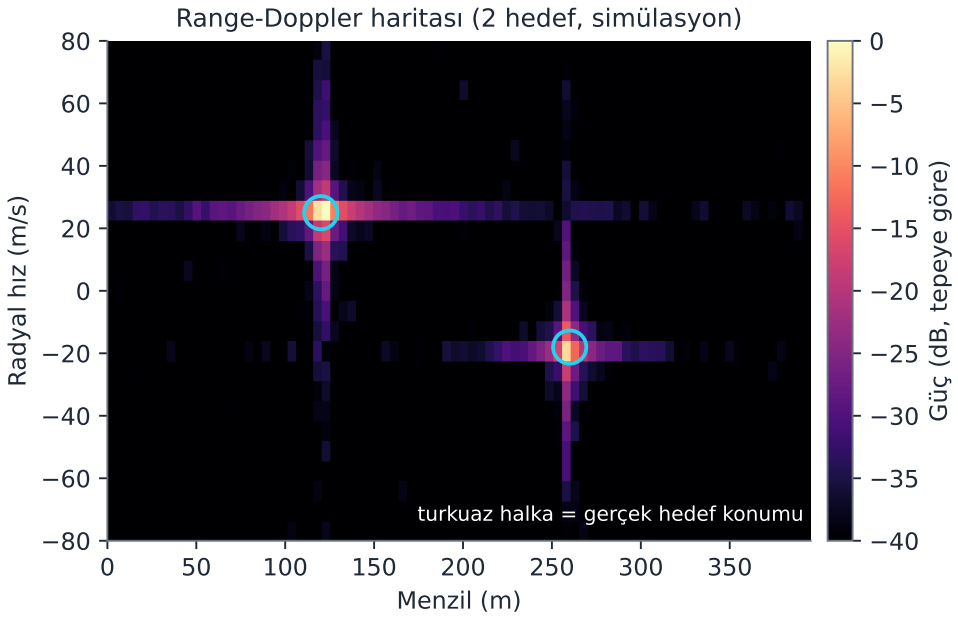
Neden önemli? Sensing sinyali, scheduler'ın dağıtabileceği kaynak havuzundan düşer. Yani sensing bir *scheduler problemi*dir: sensing kaynaklarını önceden mi ayırırız, yoksa yük arttıkça mı küçültürüz?

2.4 OFDM radar: yankıdan menzil ve hız

OFDM sinyali (5G'nin kullandığı) bir radar olarak da çalışır. Fikir şu:

1. gNB bilinen bir OFDM sinyali gönderir (X).

2. Hedeften yansıyan sinyal (Y) gecikmiş ve Doppler kaymış olarak geri gelir.
3. $H = Y/X$ (kanal tahmini) hesaplanır. Alt taşıyıcılar arasındaki **faz eğimi gecikmeyi (menzili)**, semboller arasındaki **faz eğimi Doppler'i (hızı)** verir.
4. Bir **2 boyutlu FFT** bu iki eğimi bir **range-Doppler haritasına** çevirir. Hedef haritada bir tepe noktası olarak görünür.

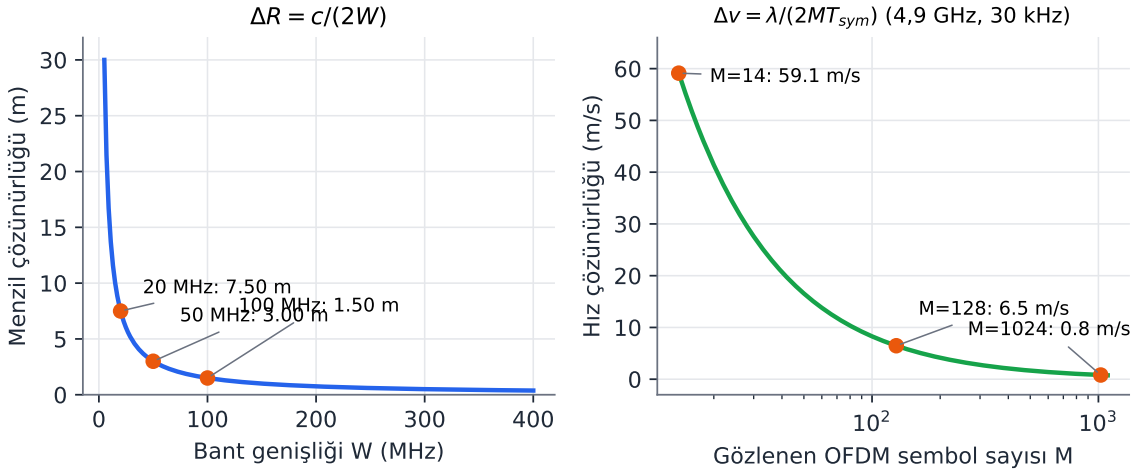


Şekil 8. Simülasyon: iki hedefli (120 m, +25 m/s ve 260 m, -18 m/s) bir OFDM radarın range-Doppler haritası. Parametreler TR 38.765 Ek A ile uyumlu: 4,9 GHz, 30 kHz SCS, 1024 alt taşıyıcı ($\approx$ 30,7 MHz), 128 sembol. Halkalar gerçek konumdur; tepeler halkaların içine düşer. Betik: `ogretici/betikler/make_figs.py`.

Çözünürlük, iki hedefi ayırt etme yeteneğidir ve iki şeye bağlıdır:

$$\Delta R = \frac{c}{2W}, \qquad \Delta v = \frac{\lambda}{2\,M\,T_{sym}} \tag{1}$$

W: sensing sinyalinin bant genişliği, M: gözlenen OFDM sembol sayısı, T_{sym}: sembol süresi, λ: dalga boyu. Bu denklemler Lin [2]'in IV. bölümünde ve Dickerson vd. [6]'in Tablo II'sinde de geçer.



Şekil 9. Çözünürlük, sensing'e verdiğimiz kaynakla belirlenir. **Daha çok bant (W) $\rightarrow$ daha ince menzil. Daha çok sembol (M) $\rightarrow$ daha ince hız.** Bu yüzden "sensing'e kaynak vermek" sensing kalitesinin doğrudan sebebidir.

Kendi gözünle gör

`kurulum\sionna_dogrulama.py` dosyasını çalıştırın. Gerçek hedef 120 m ve 25 m/s'ydi; betik hedefi 122,1 m ve 25,9 m/s olarak buldu. Hata, çözünürlük hücresinin (4,88 m, 6,47 m/s) içinde. **Soru:** bant genişliğini 4 kat artırsak menzil hatası ne olur? (Cevap: yaklaşık 4 kat azalır, formülden.)

2.5 Sensing sinyali olarak ne kullanılır?

Sensing için yeni bir sinyal icat etmek yerine, 5G'de zaten olan ve alıcının *önceden bildiği* sinyaller kullanılır (bilinen bir sinyal olmazsa $H = Y/X$ hesaplanamaz):

Sinyal	Avantaj	Sorun
PRS (DL)	Geniş bant, seyrek yapılandırılabilir, standart (Rel-16). Radar için en çok çalışılan aday [9, 8, 7].	Comb yapısı *hayalet hedef (ghost target)* üretebilir [7].
DMRS (DL, PDSCH içinde)	Veri sinyaliyle birlikte gelir: sensing neredeyse bedava.	Seyrek, yalnızca veri olan yerde var.
SRS (UL)	Telefon gönderir; bistatic/pasif sensing mümkün [11].	Telefonun gücü ve zamanlaması sınırlı.
Özel sinyal (örn. ZC)	Tasarım serbest, iyi korelasyon.	Standartta yok; ek kaynak ister [6].

OCUDU'da PRS zaten var (Bölüm 5): hem MAC'te zamanlayıcısı hem PHY'de üretici kodu mevcut. Bu, projenin en önemli teknik bulgularından biridir.

2.6 Metrikler: neyi ölçüyoruz?

İki taraf için iki metrik ailesi:

Taraf	Metrik	Nereden?
Sensing	Kaçırma olasılığı P_{md} (hedef var, bulunamadı)	TR 38.765 §4.1
	Yanlış alarm Tip 1 P_{f1} (hedef yok, tespit var)	TR 38.765 §4.1
	Yanlış alarm Tip 2 P_{f2} (tespit var, ama gerçek hedefle eşleşmiyor)	TR 38.765 §4.1
	Yatay/dikey konum hatası, hız hatası	TR 38.765 §4.1
Haberleşme	Throughput, gecikme, paket kaybı, adalet (fairness)	Capozzi vd. [17] §V
	Kaynak kullanımı	ns-3, OCUDU çıktıları
Ödünleşim	**Sensing kaynak oranı** α (Tip 1,2,3 kaynaklar, Seçenek 1 ve 2)	TR 38.765 §6.2

TR 38.765'in hedefleri (Tablo 4.2-1): $P_{md}, P_{f1}, P_{f2} \leq \%5$, yatay ve dikey konum hatası 10 m (%90 güvenle), hız hatası 5 m/s (%90 güvenle). Bu eşikler, sensing tarafında "yeterince iyi"nin ne demek olduğunu söyler.

3 3GPP standartları: ne söylüyorlar, biz neyi alıyoruz?

3.1 3GPP nedir, bu projede neden önemli?

3GPP, mobil ağ standartlarını yazan kuruluştur. 5G'nin her özelliği "Release" (sürüm) adı altında sürümlenir. Bu projede standart üç iş yapar:

1. Sensing için **ne ölçüleceğini** (metrikler) ve **hangi eşiklerin** yeterli sayılacağını söyler.

2. Simülasyonda hangi **senaryo ve parametreleri** kullanacağımızı verir (ki sonuçlarımız literatürle karşılaştırılabilsin).
3. Sensing'in ağ mimarisinde **nereye oturacağını** tarif eder (SenF, NsAP).

3.2 ISAC'ın standartlardaki yolculuğu



Şekil 10. ISAC'ın 3GPP'deki gelişimi. Kaynak: Lin [2, 3].

Biz Rel-20 / 5G-Advanced hattındayız: gNB'nin kendi sinyalini kullanarak (tek istasyon) İHA tespiti. Proposal'ın başvurduğu referans da budur.

3.3 TR 38.765'e rehber (v20.0.0, 2026-06)

Tüm raporu okumanıza gerek yok. Şu tablo hangi bölümde ne olduğunu ve neden önemli olduğunu söyler:

Bölüm	İçerik	Bizim için önemi
4.1	Performans metrikleri ve formülleri (P_{md}, P_{f1}, P_{f2}, konum/hız hatası)	Sensing metriklerimizi *buradan alırız*; kendi tanımımızı uydurmayız.
4.2	Hedefler: $P_{md}, P_{f1}, P_{f2} \leq \%5$; 10 m ve 5 m/s (%90)	Sensing "yeterince iyi" eşiği.
6.2	**Sensing kaynak oranı:** Tip 1/2/3 kaynaklar, Seçenek 1 ve 2	**Overhead'i nasıl tanımlayacağımızın standart cevabı.**
6.2, Tablo 6.2-1	Baseline 1 (52 dBm, 80 dB izolasyon), Baseline 2 (37 dBm, 65 dB)	Simülasyonda kullanacağımız referans senaryolar.
5	Ölçüm raporlama seviyeleri A, B, C, D	Sensing verisinin ağ taşıma yükünü hesaplamak için.
7–8	Ağ mimarisi: SenF, NsAP, sensing başlatma/rapor prosedürleri	OCUDU'da sensing'in nereye takılacağı.
9	Sonuç: Seviye A desteklenmedi, Seviye C kabul, SCTP tabanlı protokol	Tasarım kararlarını gerekçelendirmek için.
Ek A	Değerlendirme varsayımları	Simülasyon parametre tablosu.

3.3.1 Değerlendirme senaryosu (Ek A)

Parametre	Değer
Senaryo	UMa-AV (şehir makro hücresi, hava aracı hedefi), ikili mesafe 500 m
Taşıyıcı / bant / SCS	4 veya 4,9 GHz (FR1) / 100 MHz / 30 kHz
Yerleşim	7 makro site, her sitede 3 sektör
Anten (gNB)	8×8×2 eleman, Tx ve Rx
gNB gücü	37 veya 52 dBm
Anten izolasyonu (Tx → Rx)	65 veya 80 dB
Hedef	Küçük İHA (0,3×0,4×0,2 m), sektör başına $N = 5$, yükseklik 25–300 m, yatay hız 0–180 km/s
Sensing modu	gNB tek istasyon (monostatic), **Tx ve Rx aynı anda çalışır**
Öz-girişim (self-interference)	Artık sızıntı gürültü olarak modellenir ($-94 + X$ dBm, 100 MHz'de)

Öz-girişim: monostatic sensing'in en zor kısmı

Aynı gNB hem gönderip hem dinlediği için kendi vericisinin sızıntısı, İHA'dan dönen çok zayıf yankıdan *birçok büyüklük mertebesi* (orders of magnitude) güçlü olabilir [2]. Bu yüzden standart, anten izolasyonunu (65 veya 80 dB) ve artık sızıntıyı değerlendirmenin *parçası* yapıyor. Lin [2] bunu IV.B bölümünde "monostatic gNB sensing'in tanımlayıcı zorluğu" diye anlatır. Bizim ns-3 ve OCUDU çalışmamız öz-girişimi modellemez; yalnızca *kaynak tüketimine* bakar. Bunu raporda açıkça sınırlılık olarak yazmalıyız.

3.3.2 Sensing kaynak oranı (Bölüm 6.2): bizim ana metriğimiz

Standart üç tür kaynak tanımlar:

- **Tip 1:** sensing sinyali için kullanılan kaynaklar.
- **Tip 2:** Tip 1'in, aynı zamanda haberleşme için de kullanılan kısmı (olmayabilir).
- **Tip 3:** sensing sinyali taşımayan ama sensing işlemi yüzünden haberleşmeye de açılamayan kaynaklar.

İki seçenekle oran hesaplanır (hepsi tüm DL+UL kaynaklarına bölünür):

$$\text{Seçenek 1: } \frac{T_1 + T_3}{\text{toplam}}, \qquad \text{Seçenek 2: } \frac{T_1 - T_2 + T_3}{\text{toplam}} \tag{2}$$

Tip 2 yoksa iki seçenek aynıdır. Biz ilk aşamada bunu kullanırız: OCUDU'daki PRS rezervasyonu Tip 1'dir (veri taşımaz, Tip 2 yok) ve Tip 3'ü de gözleyebiliriz (Bölüm 5'daki "kaybolan slot" bulgusu tam olarak bir Tip 3 benzeri etkidir).

3.3.3 Ölçüm raporlama seviyeleri (Bölüm 5)

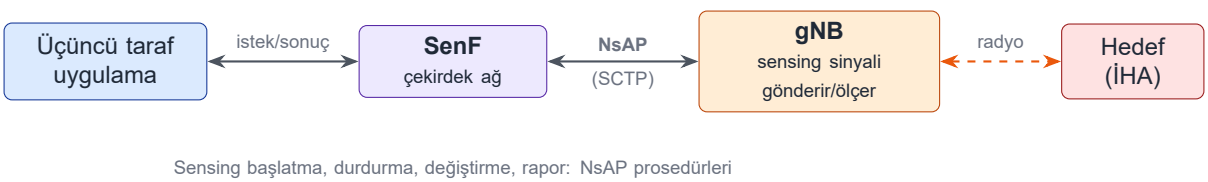
Sensing sonuçları gNB'den çekirdek ağdaki sensing işlevine *ne ayrıntıda* gönderilir? Ayrıntı arttıkça taşıma yükü artar:

Seviye	İçerik	Taşıma yükü	TR 38.765 kararı
A	Ham veri (anten başına, sembol başına kanal tahmini)	Çok yüksek	**Desteklenmedi**
B	Gecikme/Doppler/açı profilleri	Yüksek	Değerlendirildi
C	**Tespit edilen yol/nokta başına** (menzil, hız, açı, güç, güven)	Orta	**İleri normatif çalışma için kabul**
D	Nesne seviyesi (konum, hız, sınıf, iz kimliği)	Düşük	Gizlilik/güvenlik çözülürse

Lin [2] Tablo 2 aynı dört soyutlamayı (örnek, profil, tespit noktası, nesne) "transport yükü" ve "füzyon" açısından karşılaştırır.

3.4 Mimari: SenF ve NsAP

Rel-20 mimarisi, çekirdek ağa yeni bir mantıksal işlev ekler: **SenF (Sensing Function)**. Görevleri: sensing servisini yetkilendirmek, sensing yapacak gNB'yi seçmek, parametreleri vermek, ölçümleri toplamak ve dış uygulamalara sunmak [2]. gNB ile SenF arasındaki konuşma için ise yeni bir protokol önerilir: **NsAP**.



Şekil 11. Rel-20 ISAC mimarisinin sadeleştirilmiş görünümü (TR 38.765 bölüm 7–8; Lin [2] bölüm III.A).

NsAP prosedürleri (TR 38.765 bölüm 8): **Sensing Initiation** (istek/cevap/hata), **SenF-initiated Sensing Abort**, **SenF-initiated Sensing Modification**, **gNB-initiated Sensing Failure indication**, **Sensing Report**; yönetim tarafında **Ns Setup**, **RAN Configuration Update**, **Error indication**. Sensing verisi için üç protokol yığını değerlendirildi; **SCTP tabanlı olanı (Seçenek 1)** önerildi.

Bizim için anlamı: OCUDU gerçek bir 5G gNB uygulamasıdır; NsAP gibi bir arayüz henüz yok. Bunu eklemek "tam bir sensing özelliği" olurdu, ama proje kapsamı için fazla büyük. Biz bunu *tasarım seviyesinde* (bir diyagram ve bir arayüz taslağı) ele alacağız, kodlamayı değil.

3.5 Kanal modeli: hedef, arka plan ve yerel kuplaj

Rel-19, TR 38.901'e *sensing'e özgü* bir uzantı ekledi [2, 4]. Fikir: bir sensing kanalı üç parçaya ayrılır:

- **Hedef kanalı:** gNB → hedef → gNB yolu ve hedefin yansıtma özelliği (RCS).
- **Arka plan kanalı:** bina, ağaç, araba gibi *hedef olmayan* yansıtıcılar (clutter).
- **Yerel kuplaj:** vericiden alıcıya doğrudan sızıntı (öz-girişim).

Ayrıca RCS sabit bir sayı değil; açıya, polarizasyona ve frekansa göre değişir. Bir İHA, yalnızca biraz konum değiştirince RCS çukurlarına girip *kaybolabilir* [2].

3.6 Hangi standart belgelerini bilmeliyiz?

Belge	Ne için
TR 38.765	ISAC çalışması, **ana referansımız** (indirildi, `makaleler/` klasöründe).
TR 38.901	Kanal modeli; Rel-19 ISAC uzantısı (hedef/arka plan kanalı).
TS 38.211 §7.4.1.7	**PRS sinyalinin tanımı** (OCUDU kodundaki yorumlarda bu bölüme atıf var).
TS 38.455 §9.2.44	PRS kaynak kümesi yapılandırması (NRPPa); OCUDU'nun PRS parametreleri bunu izler.
TR 36.777	İHA kanal modeli (TR 38.765 Ek A bu belgenin B.1.3 maddesine atıf yapar).

Standart belgesi nasıl okunur?

Standartlar roman değil, *başvuru* metinleridir. Baştan sona okumayın. **(1)** İçindekilerden ihtiyacınız olan bölümü bulun, **(2)** tanımları ve tablolardaki sayıları okuyun, **(3)** "NOTE" kutularına özellikle bakın, çünkü kısıtlar ve belirsizlikler orada yazılıdır. TR 38.765'te de projenin boşluğunu gösteren cümle bir bölüm girişinde duruyordu.

4 Üç araç: ne işe yarar, biz ne yaptırırız?

4.1 Sionna: fiziksel katman ve ray tracing

Sionna, NVIDIA'nın açık kaynak, TensorFlow tabanlı fiziksel katman kütüphanesidir [15]. İki parçası var:

- **Sionna (çekirdek):** OFDM, MIMO, kanal kodlama, kanal modelleri (bit hata oranı eğrileri gibi bağlantı seviyesi simülasyon).
- **Sionna RT:** 3B bir sahnede (bina, yol) ışın izleme ile *gerçek yer-özel* kanal üretir [16]. Sahnede bir verici ve alıcı koyup hangi yolların oluştuğunu hesaplar.

Projedeki rolü: sensing'in *ne kadar iyi* çalıştığını göstermek. Yani şu eğriyi çıkarmak: "sensing'e ayrılan bant genişliği ve sembol sayısı arttıkça menzil/hız doğruluğu ve tespit olasılığı nasıl değişir?" Bunun ilk

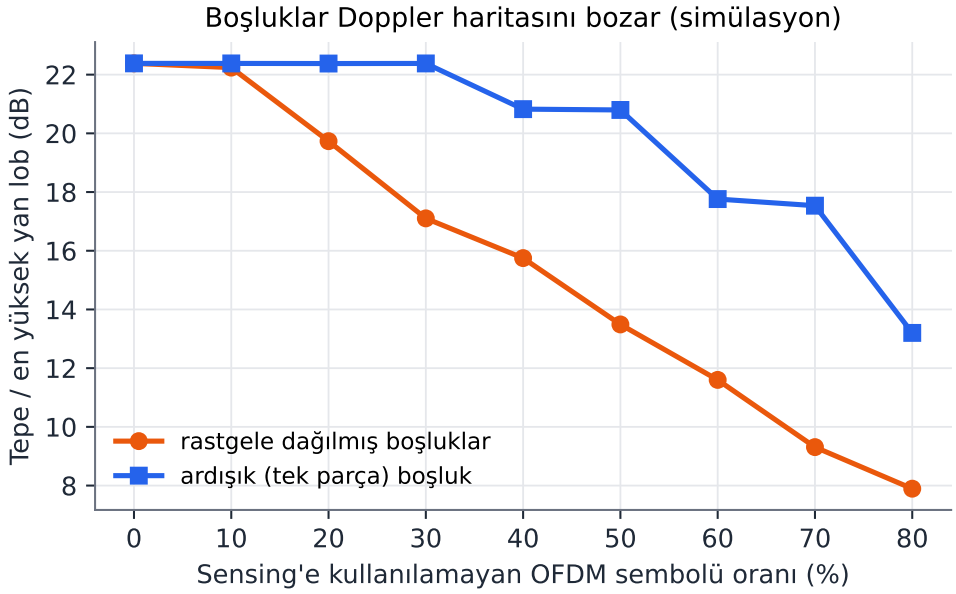
adımını Bölüm 2.4'de zaten attık (range-Doppler haritası, çözünürlük). Resmî Sionna RT öğreticileri: *Introduction to Sionna RT*, *Tutorial on Mobility*, *Tutorial on Radio Maps*, *Tutorial on Radar Cross Section (RCS)*.

Sionna ile ilgili iki tuzak (ölçülen)

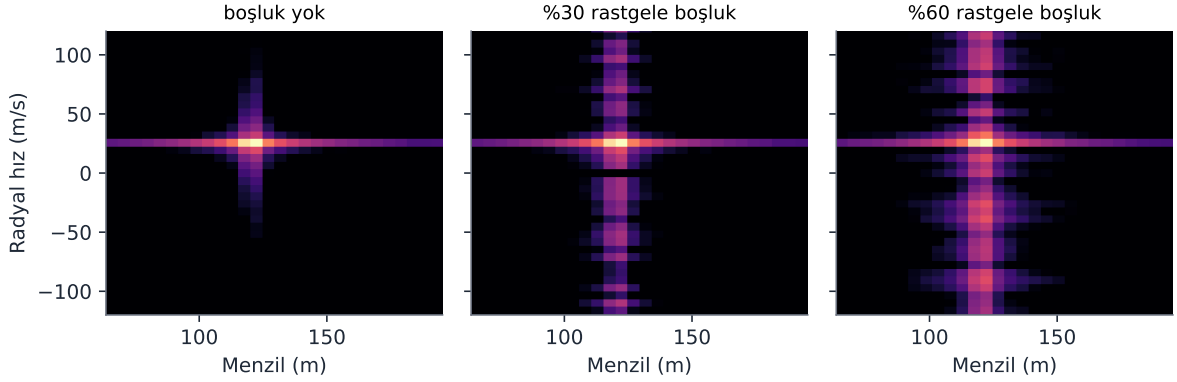
(1) Python 3.11'de `pip install sionna` en yeni sürümü, **Sionna 2.2.0**'ı seçiyor (kurulum sırasında PyTorch indirdi; yani muhtemelen farklı bir API, bunu incelemedik). Betiklerimiz 1.2.2 için yazıldığından `sionna==1.2.2` sabitliyoruz. **(2)** Sionna RT, Windows'ta CPU üzerinde çalışmak için **LLVM** ister; yoksa `LLVM-C.dll bulunamadı` hatası verir (`kurulum/03-sionna-windows.md` bölüm 5).

4.1.1 Haberleşme zamanlaması sensing'i bozar mı? (basit deney)

Haberleşme scheduler'ı, sensing sinyalinin bazı OFDM sembollerini başka kullanıcılara verirse, sensing alıcısı sembol dizisinde *boşluklar* görür. Lin [2] bunu şöyle özetler: "sensing fırsatları arasındaki boşluklar Doppler işlemeyi bozar". Bunu tek hedefli bir OFDM radarla denedik:



Şekil 12. Simülasyon (basit model, 12 koşunun ortalaması): OFDM sembollerinin yüzde kaçı sensing'e kullanılamıyorsa range-Doppler haritasında tepe/yan lob oranı (ne kadar yüksekse o kadar iyi) nasıl düşüyor? Rastgele dağılmış boşluklar, aynı oranda ardışık boşluktan *daha çok* zarar veriyor.



Şekil 13. Aynı deneyden haritalar. Boşluk arttıkça hedefin etrafında hız ekseni boyunca *sahte yan loblar* oluşur; hedefin gerçek hızı bulanıklaşır.

Bu bir oyuncak model

Bu deney *tek hedef, sabit SNR, ideal kanal, gerçekçi olmayan boşluk dağılımları* ile yapıldı. Sonucun *yönü* (boşluk → kötüleşme) literatürle uyumludur, ama sayıları (dB) gerçek bir sistem için geçerli değildir. Raporda bu deneyi "kavramı gösteren simülasyon" olarak sunun, "sonuç" olarak değil.

4.2 ns-3 ve 5G-LENA: ağ seviyesi simülasyon

ns-3, ayrık olay tabanlı bir ağ simülatörüdür. **5G-LENA** (`nr` modülü) onun üstüne eklenen 5G NR modülüdür: gerçek protokol yığınını (PHY, MAC, RLC, PDCP, çekirdek ağ) 3GPP'ye uygun olarak simüle eder [14]. Kalibrasyonu 3GPP referans sonuçlarına karşı yapılmıştır [13]. Büyük bir avantajı: *çok kullanıcılı* bir ağda scheduler etkisini görebilmemiz.

Nasıl çalışır? Bir C++ senaryo dosyası ("örnek") düğümleri, anten dizilerini, bantları ve trafik akışlarını tanımlar; simülatör çalışır ve her akış için throughput, gecikme, jitter ve paket sayılarını yazar.

```
Ubuntu (WSL2): ns-3 çıktısının son satırları

$ ./ns3 run "cttc-nr-demo --ueNumPergNb=5"
  Mean delay:  24.555788 ms
  Mean jitter:  0.121567 ms
  Rx Packets: 5534
Flow 5 (1.0.0.2:49157 -> 7.0.0.6:1235) proto UDP
  Tx Packets: 6000
  Tx Bytes:   7680000
  TxOffered:  102.400000 Mbps
  Rx Bytes:   7087360
  Throughput: 94.498133 Mbps
  Mean delay:  24.440323 ms
  Mean jitter:  0.121564 ms
  Rx Packets: 5537
  Mean flow throughput: 43.930624
  Mean flow delay: 9.970234
```

Şekil 14. Gerçek çıktı: `./ns3 run "cttc-nr-demo --ueNumPergNb=5"` komutunun son satırları (Ubuntu/WSL2). Her akış için throughput, gecikme, jitter ve alınan paket sayısı.

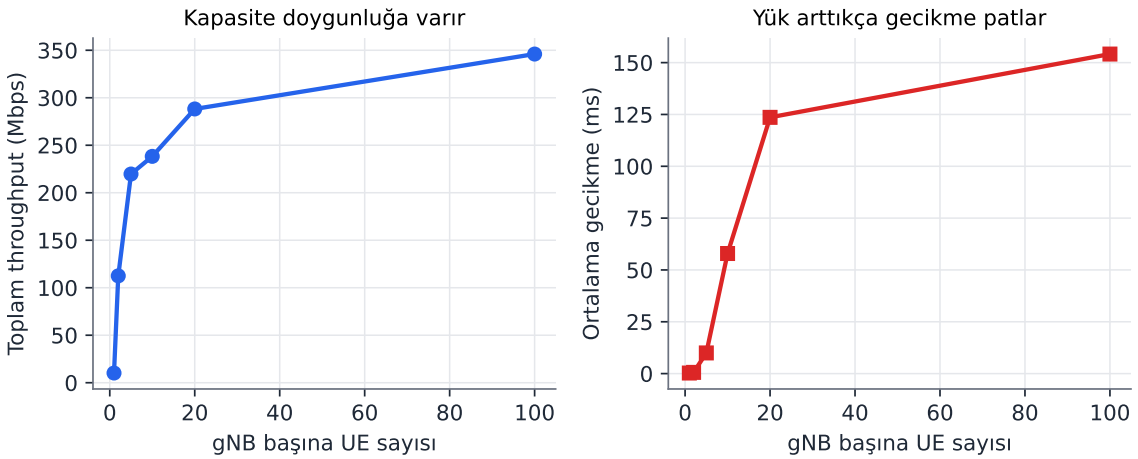
ns-3 ölçümü: UE sayısına göre throughput ve gecikme

`cttc-nr-demo` örneğini `--ueNumPergNb` = 1, 2, 5, 10, 20 ve 100 ile çalıştırdık (5G-LENA v5.1, ns-3.48, 1 s simülasyon, tek koşu). Sonuçlar:

UE/gNB	akış	toplam throughput (Mbps)	ort. gecikme (ms)
1	1	10,2	0,28
2	2	112,5	0,59
5	5	219,7	9,97
10	10	238,3	57,9
20	20	288,2	123,6
100	100	346,0	154,1

Yorum: Örnek, UE'leri iki gruba böler: bir kısmına düşük gecikmeli 10,24 Mbps trafik, kalanına 102,4 Mbps "best effort" trafik verir (5 UE'de çıktıda 3'üne 10,24 ve 2'sine 102,4 Mbps verildiğini gördük). Yük arttıkça toplam kapasite doygunluğa varır ve gecikme patlar.

100 UE: bu koşu **26 dakika** sürdü (20 UE'de birkaç dakikaydı), yani süre UE sayısıyla doğrusal artmıyor. Sunulan toplam yük 5632 Mbps iken teslim edilen yalnızca 346 Mbps (%6,1). Çarpıcı olan, çok farklı talepli akışların *neredeyse eşit pay almasıdır*: 10,24 Mbps isteyen 49 akış **3,40 Mbps**, 102,4 Mbps isteyen 50 akış **3,58 Mbps** aldı. Gecikme ise farklı: düşük gecikmeli sınıf ortalama 17 ms, "best effort" sınıf 292 ms. Bir akış (No. 47) hiç paket alamadı; nedenini incelemedik. Bu, *kaynaklar azalırsa* ne olacağının bir önizlemesidir: sensing de benzer şekilde kapasite alır.



Şekil 15. Yukarıdaki tablo grafik olarak (gerçek 5G-LENA ölçümü; veri: `ogretici/veri/ns3_ue_sweep.csv`).

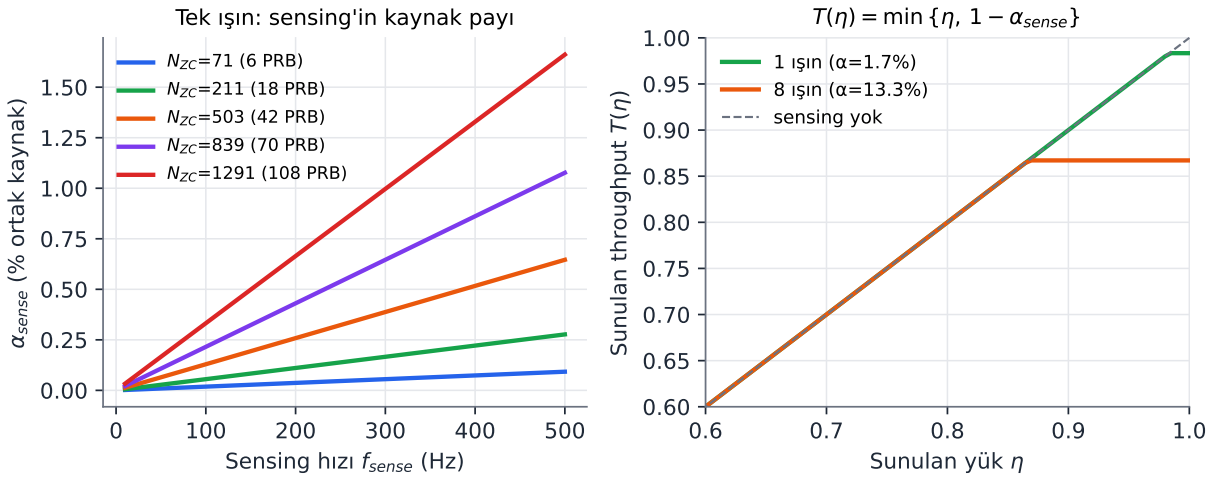
4.2.1 Analitik model: sensing kaynak payı ve throughput

Dickerson vd. [6], sensing'in kaynak tüketimini şöyle modeller (denklem 9–12):

$$\alpha_{sense} = \frac{N_b}{1 - OH} \cdot \frac{f_{sense}}{N_{sym/sn}} \cdot \frac{N_{PRB,sense}}{N_{PRB,toplam}}, \qquad T(\eta) = \min\{\eta,\ 1 - \alpha_{sense}\} \tag{3}$$

N_b: sensing ışın sayısı, f_{sense}: saniyedeki sensing sayısı, $N_{PRB,sense}$: bir sensing dalga biçiminin kapladığı PRB, OH: sabit NR kontrol yükü, η: sunulan haberleşme yükü (normalize).

Makalenin parametreleriyle ($N_{PRB,toplam} = 270$, $OH = 0{,}14$, 14 000 sembol/s, $N_{ZC} = 1291 \Rightarrow 108$ PRB, $f_{sense} = 500$ Hz) **makalenin verdiği iki sayıyı elle yeniden hesapladık**: tek ışın için $\alpha = \%1{,}66$ ve sekiz ışın için $\alpha = \%13{,}29$ (makalede de aynı değerler).



Şekil 16. Dickerson vd. modelinin yeniden üretimi. Sol: sensing hızı arttıkça kaynak payı. Sağ: sunulan throughput. 8 ışında yüksek yükte throughput %86,7'de tavan yapar.

4.2.2 5G-LENA'ya sensing yükünü nasıl koyarız? (açık tasarım sorusu)

5G-LENA'da hazır bir sensing modeli **yok**. Sensing yükünü simülasyonda temsil etmenin üç aday yolu var; hangisinin *mümkün ve yeterince doğru* olduğunu **pilot görev** cevaplayacak:

Yol	Artısı	Riski

a	Bant genişliğini $(1-\alpha)$ ile küçült	Çok basit, kodlama yok	Sensing'in "zamansal" yapısını (periyodik, atımlı) hiç temsil etmez
b	Sensing'i yüksek öncelikli periyodik bir trafik akışı olarak ekle	Atımlı yapı ve scheduler etkisi görünür	Gerçek PRB rezervasyonu değil, trafik: yaklaşım
c	Scheduler'ı değiştirip PRB'leri periyodik rezerve et	En gerçekçi (OCUDU'daki PRS gibi)	C++ içinde geliştirme ister; en çok emek

Önerimiz önce **(a) ve (b)** ile başlamak ve **(c)'yi, OCUDU ölçümüyle (Bölüm 5) tutarlılık kontrolü olarak** ancak zaman kalırsa denemek.

4.2.3 Sionna'dan ns-3'e kanal köprüsü: SioLENA

Ropitault vd. [12], Sionna RT'nin ürettiği çok yollu kanalı (gecikme, genlik, faz, açı) CSV olarak alıp 5G-LENA'nın kanal modeline besleyen **SioLENA** modülünü yayımladı (`github.com/usnistgov/siolena`; CSV üretici: `github.com/wineslab/sionna-channel-generator`). Böylece *yer-özel kanallı* ağ simülasyonu mümkün.

SioLENA'yı kullanmadan önce üç şeyi bil

(1) SioLENA deposu *kendi içinde* ns-3.44 ve 5G-LENA v4.0 taşır; bizim ns-3.48 + v5.1 kurulumumuza eklenmez, *ayrı klasörde ayrı derlenir*. **(2)** CSV üretici araç **Sionna 0.19.2** ve **Python 3.11+** ister; bizim Sionna 1.2.2 ortamıyla çakışır, ayrı sanal ortam gerekir. **(3)** Bunların hiçbirini henüz kurup denemedik. Bu bir *ikinci aşama* işidir.

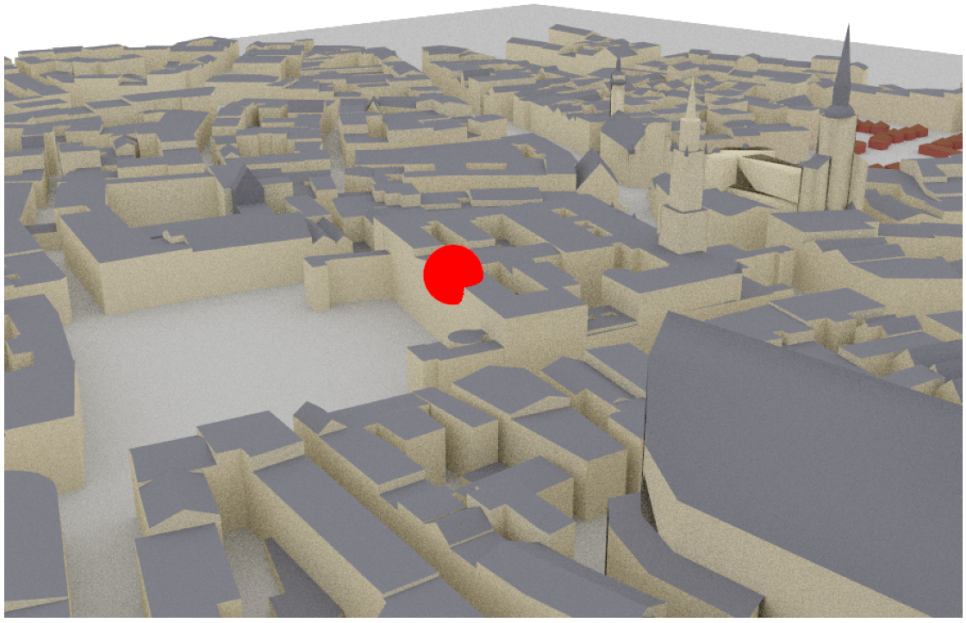
4.3 Üç araç ne işe yarar? Tek bakışta

Araç	Gerçek hayatta neyin yerine geçer?	Ne yaparsın?	Bizim sorumuza cevabı
Sionna	Bir radar/radyo laboratuvarı ve ölçüm arabası: sinyalin şehirde nasıl yayıldığını ve radarın hedefi nasıl gördüğünü hesaplar	Sahne kur, gNB koy, kapsama haritası çıkar, İHA uçur, radar işle	Sensing'e verilen kaynak arttıkça *sensing ne kadar iyi* oluyor?
ns-3 / 5G-LENA	Bir şehir ölçeğinde ağ: yüzlerce telefonun aynı baz istasyonunu paylaştığı bir trafik laboratuvarı	Düğümler ve trafik tanımla, çalıştır, throughput ve gecikmeyi oku	Kaynak sensing'e gidince *çok kullanıcılı bir ağ* ne kadar etkileniyor?
OCUDU	Gerçek bir baz istasyonunun yazılımı: radyo hariç her şey gerçek kod	gNB'yi çalıştır, yapılandırmayı değiştir, scheduler'ın kararlarını log'dan izle	*Gerçek kodda* sensing sinyali (PRS) haberleşmeden ne kadar götürüyor?

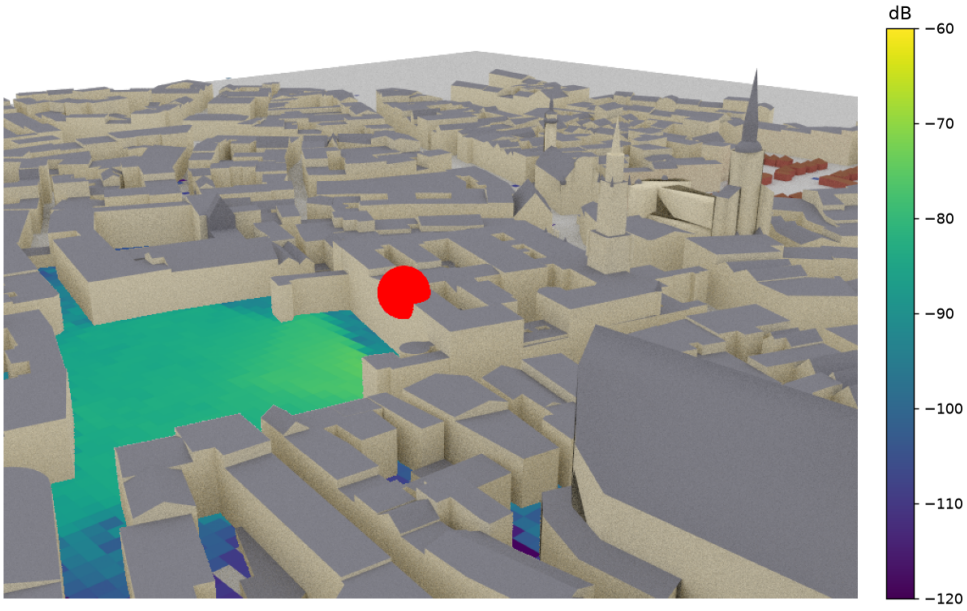
Üçü aynı sorunun üç yüzü: **sensing yüzdesi α arttıkça kazanç (Sionna) ve bedel (ns-3, OCUDU) nasıl değişir?**

4.4 Görsel tur: Sionna ile şehirde ISAC projesi

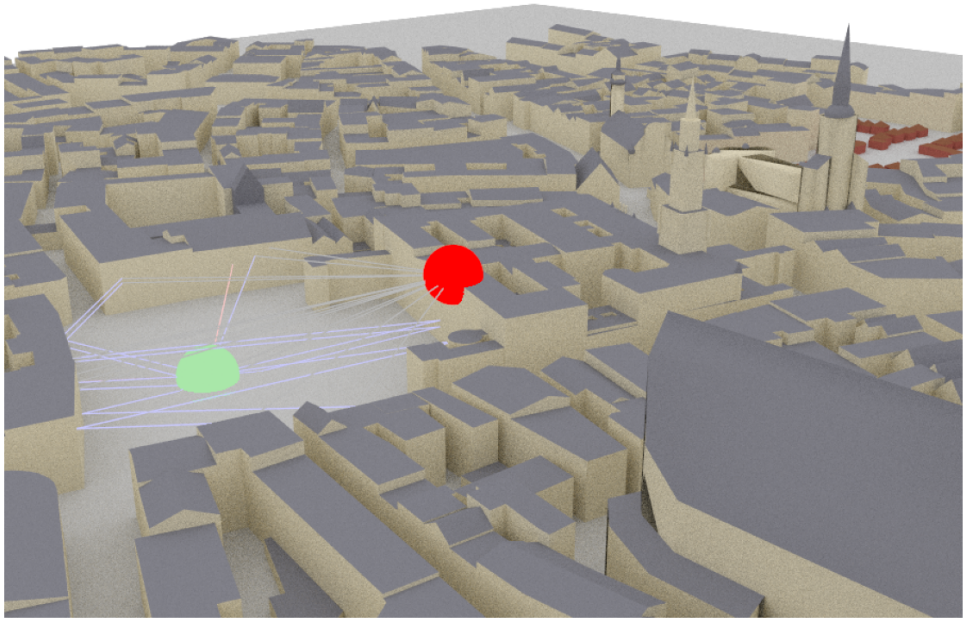
Bu bölüm, Sionna'nın *ne işe yaradığını* gözle görmek için hazırlandı. Proje bir şehirde (Münih, Sionna'nın yerleşik sahnesi) şunları yapar: **(1)** gNB'yi bir çatıya koy, **(2)** sinyalin şehirde nereye ulaştığını hesapla (kapsama), **(3)** bir telefona giden yolları göster, **(4)** bir İHA uçur ve radarın onu izlemesini sağla, **(5)** sensing'e ayrılan kaynak arttıkça sonuçların nasıl değiştiğini ölç. Kodlar: `ogretici/sionna_sehir/` (`rt_sehir.py`: Sionna RT; `radar_takip.py`: radar ve takip). Toplam çalışma süresi birkaç dakikadır.



Şekil 17. Adım 1: şehir ve gNB. Münih'in 3B modeli; kırmızı küre gNB (`[8,5; 21; 27]` m, bir çatı kenarı), 4,9 GHz, 37 dBm. Sionna RT ile üretildi (Mitsuba yol izleyici, CPU).

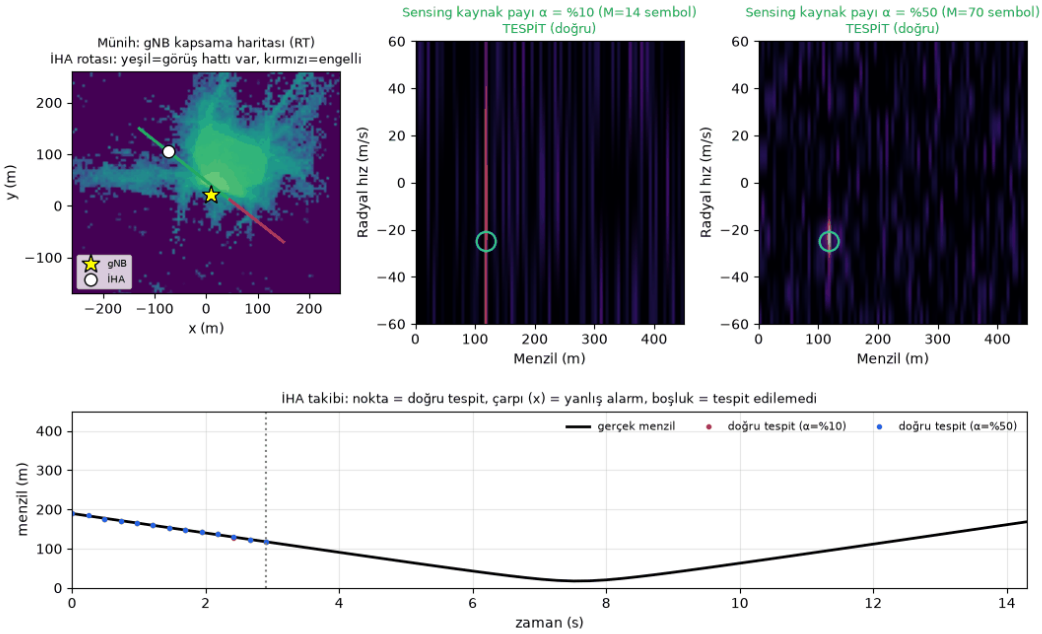


Şekil 18. Adım 2: kapsama haritası. Renk, her noktadaki yol kazancı (dB). Sinyal en çok gNB'nin baktığı açık meydana ulaşıyor; binaların arkasında ve dar sokaklarda zayıf. Işın izleme bunu binaların yansıma ve gölgelemesiyle hesaplıyor.

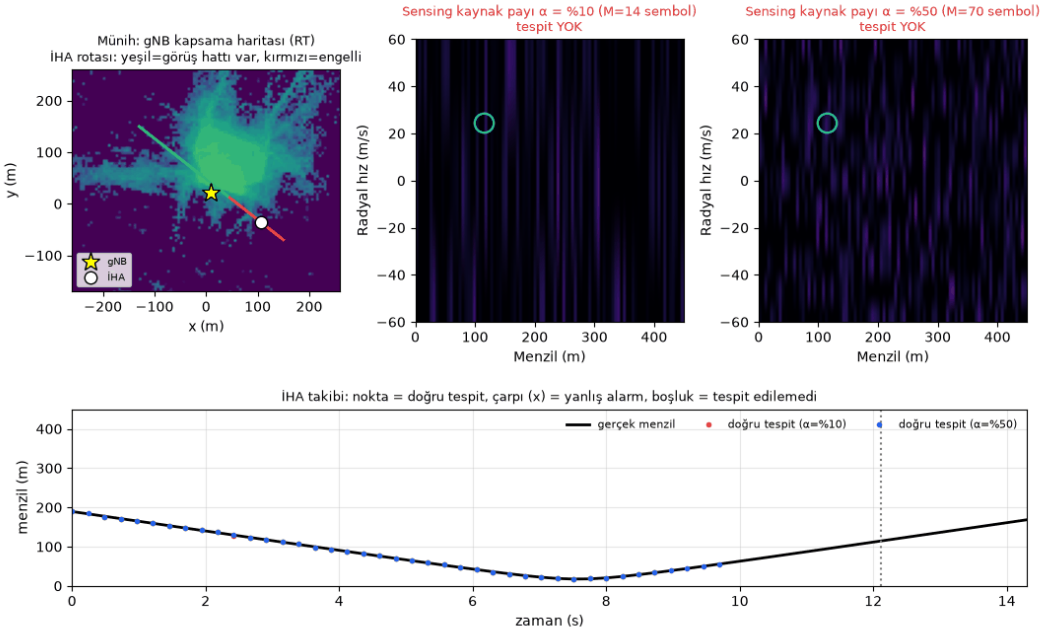


Şekil 19. Adım 3: bir telefona giden yollar. Yeşil küre bir kullanıcı (`[45; 90; 1,5]` m). Işık çizgiler, sinyalin gNB'den ona gittiği yollardır: bir doğrudan yol ve binalardan yansıyanlar. Sionna 13 geçerli yol buldu. En güçlü yol doğrudan yol (274 ns $\approx$ 82 m, $-84{,}5$ dB), en zayıf listelenen yol -104 dB. Bu *çok yollu kanal*, kablosuz ağın neden kablolu ağdan farklı davrandığının nedenidir.

Adım 4: İHA'yı uçur ve radarla izle. Bir İHA, 20 m irtifada 356 m'lik bir rotada yaklaşık 25 m/s ile uçar (60 nokta, 14,3 s). Her noktada iki şey hesaplanır: **(i)** gNB'nin İHA'yı *görüp görmediği* (görüş hattı), bunu Sionna RT'ye *doğrudan yolu çözdürerek* buluyoruz; **(ii)** bu geometride radarın İHA'yı tespit edip edemediği. Görüş hattı 60 noktanın **37**'sinde var (%62). Gerisinde İHA bir binanın arkasında.

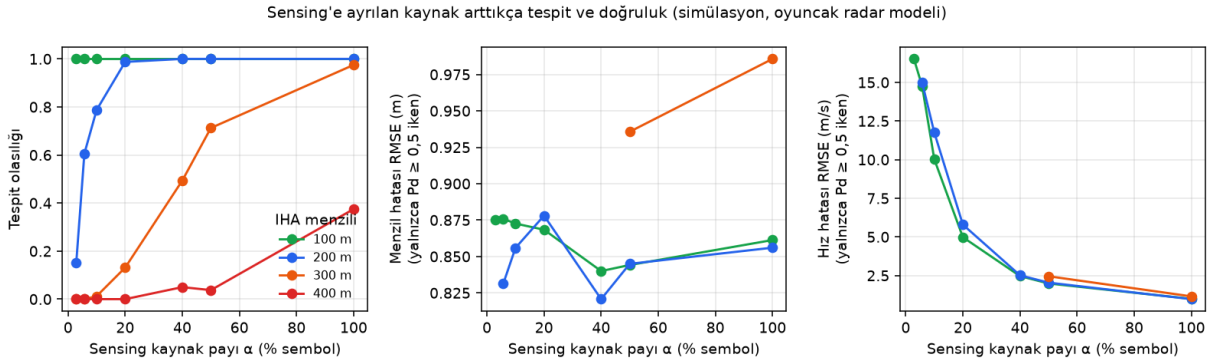


Şekil 20. Takip filmi, 1. kare (tam film: `ogretici/sionna_sehir/cikti/05_takip_filmi.gif`). Sol: harita, İHA rotası (yeşil = görüş hattı var). Orta ve sağ: iki radarın menzil-hız görüntüsü, **sensing payı %10** ve **%50**. Turkuaz halka İHA'nın gerçek konumu. **%10 payda (14 sembol) hedef hız ekseninde uzun bir çizgi gibi yayılıyor** (hız çözünürlüğü kaba), **%50 payda (70 sembol) tek bir nokta.** Alt: gerçek menzil (siyah) ve radarın bulduğu menzil.



Şekil 21. Takip filmi, engelli bölüm. İHA rotanın kırmızı kısmında, yani bir binanın arkasında. **İki radar da hedefi göremiyor** ve alttaki izde noktalar kesildi. *Sensing kaynağını artırmak engeli aşmaya yetmedi.* Bu, sensing kalitesinin yalnızca kaynağa değil, *geometriye* (görüş hattı) da bağlı olduğunu gösteriyor.

Adım 5: sensing'e ayrılan kaynak ne kazandırıyor? Sensing payını $\alpha = M/140$ tanımlıyoruz (M: 10 ms'lik çerçevede (140 sembol) sensing'e ayrılan sembol sayısı). Her (α, menzil) için 160 deneme yaptık:



Şekil 22. Sensing kaynağı ↔ başarı (Sionna tarafı). Sol: tespit olasılığı. Orta: menzil hatası. Sağ: hız hatası (orta ve sağda yalnızca $P_d \geq 0{,}5$ noktalar). Renkler İHA'nın gNB'den uzaklığı.

Sionna tarafından üç bulgu

(1) Yakın hedef ucuza bulunur, uzak hedef pahalıya. 200 m'de tespit %20 payda (28 sembol) 0,99'a ulaşır; **300 m'de** aynı olasılık için yaklaşık %100 pay gerekir (%40'ta 0,49, %50'de 0,71, %100'de 0,98); **400 m'de** %100 payda bile yalnızca 0,38. **(2) Menzil doğruluğu kaynak payına bağlı değil:** RMSE tüm α için yaklaşık 0,85–1,2 m kalıyor. Menzil doğruluğunu *bant genişliği* belirler (Bölüm 2.4), α değil. **(3) Hız doğruluğu α ile hızla düzelir:** RMSE yaklaşık 16 m/s'den (%3 pay) 1 m/s'ye (%100 pay) iner, $\Delta v \propto 1/M$ olduğundan. Yani **sensing kaynağı neye yarar?** Daha uzak hedefi bulmaya ve hızı daha iyi ölçmeye; menzil hassasiyetine değil.

Bu bölümdeki varsayımlar ve sınırlar (hepsi `radar_takip.py` içinde)

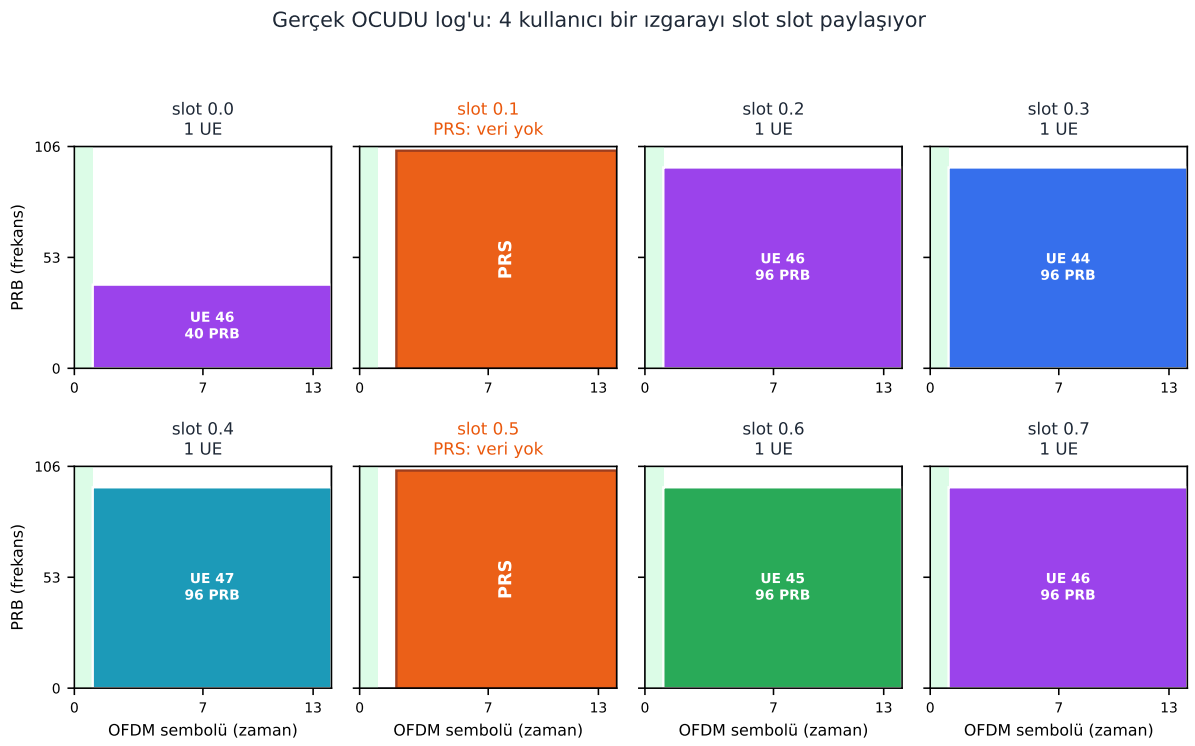
Radar kısmı *standart radar denklemiyle yazılmış oyuncak bir modeldir*, Sionna'nın RT'sinden yalnızca **geometri ve görüş hattı** geliyor. Varsayımlar: gNB gücü 37 dBm, anten kazancı 26 dBi (gönderim ve alım), İHA RCS -20 dBsm, **artık öz-girişim X = 40 dB** (TR 38.765'te "$-94 + X$", X şirket seçimi), engelli noktada yankıya **30 dB** ek kayıp, yanlış alarm olasılığı $P_{fa} = 10^{-3}$. Bu sayılar *eğilimi* gösterir; mutlak değerleri (örn. "300 m'de %100 pay gerekir") gerçek bir sistem için geçerli değildir. Öz-girişimin 40 dB seçilmesi, uzak menzildeki başarıyı doğrudan belirleyen büyük bir varsayımdır. İlk denemede yanlış alarm olasılığı sabit eşikle çok yüksek çıkıyordu (engelli bölgede sahte "tespit"); eşiği hücre sayısına göre ayarlayınca bu kayboldu (60 karede 0 yanlış alarm). Ayrıca kapsama haritası *yer seviyesi* (kullanıcı yüksekliği) içindir, İHA'nın uçtuğu 20 m için değildir.

4.5 Görsel tur: OCUDU'da çok kullanıcılı ve dilimli deney

Bu kez OCUDU'ya birden çok sahte kullanıcı (test UE) bağladık ve PRS'i açıp kapattık. Betikler: `ogretici/betikler/ocudu_c` `ocudu_cok_ue_sweep.sh`.

UE sayısı	PRS yok: toplam	UE başına	PRS her 4 slot: toplam	UE başına
1	59,0 Mbps	59,0	44,0 Mbps	44,0
2	60,0	30,0	44,0	22,0
4	60,0	15,0	44,0	11,0
8	60,8	7,6	45,1	5,6

(Veri: `ogretici/veri/ocudu_cok_ue.csv`. DL, 20 MHz, 15 kHz, son 15 saniyenin ortalaması, tek koşu.)



Şekil 23. Gerçek OCUDU log'u, 4 kullanıcı, PRS her 4 slotta bir. Her kare bir slot. **Her slotun tamamı (96 PRB) tek bir kullanıcıya veriliyor**, kullanıcılar ızgarayı yan yana değil *slot slot sırayla* paylaşıyor (mor UE 46, mavi UE 44, ...). Turuncu slotlar PRS: kimseye yer yok. Animasyon: `ogretici/sekiller/ocudu_cok_ue_filmi.gif`.

OCUDU çok kullanıcılı deneyden dört bulgu

(1) Hücre kapasitesi sabit bir havuz: kullanıcı sayısından bağımsız $\approx$ 60 Mbps. N kullanıcı bunu eşit böler ($60/N$). **(2) PRS havuzu küçültür, herkesin payı aynı oranda küçülür:** 60 $\rightarrow$ 44 Mbps ($-$%27), 4 kullanıcıda 15 $\rightarrow$ 11 Mbps. Sensing, *adil biçimde* herkesten alıyor. **(3) Paylaşım zamanda:** log'da (5873 PDSCH kaydı) bir slotta tek kullanıcı görüldü; tüm slotları tek tek denetlemedik. **(4) Dilim (slicing) sınırı bu ortamda etkisiz:** `max_prb_policy_ratio` %60 ve %30 verildi, throughput 59,0 Mbps'te kaldı. Büyük olasılıkla testmode UE'leri çekirdek ağ olmadan yaratıldığı için hiçbir dilime atanmıyor. Yani **sensing'i dilimle emüle etme fikri** testmode'da çalışmadı; gerçek UE ve çekirdek ağ ister. Kod okurken ayrıca şunu gördük: varsayılan scheduler politikası `time_qos` (`du_high_config_cli11_schema.cpp`, `derive_cell_auto_params`); eşit paylaşım bu politikanın testmode'daki davranışı.

4.6 ns-3'ü görmek: NetAnim

ns-3'ün resmî animasyon aracı **NetAnim**, düğümleri ekranda gösterir ve paketlerin hareketini oynatır. Ubuntu'da hazır paketi yok; kaynaktan derlendi (`kurulum/wsl_netanim_kur.sh`, Qt5 + CMake). Programa `AnimationInterface` eklenince bir XML üretilir ve NetAnim'de açılır (Bölüm 4.2'teki iki düğümlü örnek böyle izlendi). Pencere Windows 11'de WSLg ile masaüstüne açılır.

4.7 OCUDU: gerçek bir gNB yazılımı

OCUDU, srsRAN Project'in devamı olan, Linux Foundation altında yönetilen açık kaynak 5G CU/DU yazılımıdır (lisans: BSD-3-Clause-Open-MPI). Bir uygulamada CU-CP, CU-UP ve DU'yu birleştiren tek bir `gnb` programı vardır. Gerçek radyoya (USRP) ya da yazılımsal sahte radyoya (ZeroMQ, `ru_dummy`)

bağlanabilir.

Neden ns-3 yetmez de OCUDU da var? ns-3 *modeldir*: gerçek kod değil. OCUDU *gerçek koddur*: gerçek bir scheduler'ın, gerçek PRS sinyalini nasıl rezerve ettiğini ve throughput'u nasıl etkilediğini ölçebiliriz. Bu, sonuçlarımıza *güvenilirlik* katar. Detaylı kod haritası bir sonraki bölümde.

```
Ubuntu (WSL2): OCUDU gNB testmode (gerçek çıktı)

$ ./gnb -c ehb_p4.yml      # PRS her 4 slotta bir
--== OCUDU gNB (commit e0db566) ==--
Cell pci=1, bw=20 MHz, 1T1R, dl_arfcn=368500 (n3), dl_freq=1842.5 MHz, dl_ssb_arfcn=368410, ul_freq=1747.5 MHz
==== gNB started ===
            |-------------------DL--------------------|-----------------------------UL-----------------------------
    1   44 |  15 1.0   28    44M  750    0   0%    10M |  99.9   n/a   1   28  77.3M 1000    0   0%  81.9M      0  n/a
    1   44 |  15 1.0   28    44M  750    0   0%    10M |  99.9   n/a   1   28  77.3M 1000    0   0%  81.9M      0  n/a
    1   44 |  15 1.0   28    42M  733   17   2%    10M |  99.9   n/a   1   28  75.4M  975    0   0%  81.9M      0  n/a
    1   44 |  15 1.0   28    44M  750    0   0%    10M |  99.9   n/a   1   28  77.3M 1000    0   0%  81.9M      0  n/a
    1   44 |  15 1.0   28    44M  750    0   0%    10M |  99.9   n/a   1   28  77.3M 1000    0   0%  81.9M      0  n/a
    1   44 |  15 1.0   28    44M  750    0   0%    10M |  99.9   n/a   1   28  77.3M 1000    0   0%  81.9M      0  n/a
```

Şekil 24. Gerçek çıktı: OCUDU gNB, radyosuz (`ru_dummy`) `testmode`'da, PRS her 4 slotta bir. DL throughput 44 Mbps (PRS yok iken 59 Mbps). Sütunlar: `cqi ri mcs brate ok nok (%) dl_bs` (DL) ve `pusch rsrp ri mcs brate ok nok (%) bsr ta phr` (UL).

5 OCUDU kodu: nerede ne var, nereye ekleme yaparız?

Bu bölüm, OCUDU 26.10.0 kaynak kodunu (`release_26_10`, commit `e0db566`) gerçekten okuyarak yazıldı. Dosya yolları `~/ehb415/ocudu` altındadır.

5.1 Depo haritası

Klasör	Satır (.h+.cpp)	İçerik
`apps/gnb`	–	`gnb` programı: yapılandırmayı okur, CU/DU/RU'yu bir araya getirir.
`apps/units/...`	–	Yapılandırma şemaları (YAML anahtarları burada tanımlı).
`lib/scheduler`	53 502	**MAC scheduler** (kaynak dağıtımı). **Bizim ana ilgi alanımız.**
`lib/mac`	10 882	MAC katmanı (scheduler'ın sonuçlarını PDU'lara çevirir).
`lib/phy`	65 715	Fiziksel katman: PRS üretici, PDSCH, SRS, ... (alt klasör `signal_processors/prs`).
`lib/fapi_adaptor`	–	MAC ↔ PHY arayüzü (FAPI); PRS çevirileri (`.../pdu_translators/prs.cpp`).
`lib/du`	24 963	DU yönetimi ve yapılandırması.
`lib/cu_cp`, `lib/cu_up`	29 397 / 4 702	Merkezi birim: kontrol ve kullanıcı düzlemi.
`lib/f1ap`, `lib/e1ap`, `lib/ngap`	15 595 / – / 9 399	Arayüz protokolleri (F1 CU–DU, E1, NG çekirdek ağa).
`lib/rlc`, `lib/pdcp`	7 441 / 4 300	Üst katman protokolleri.
`lib/nrppa`, `lib/lpp`	–	**Konumlandırma** protokolleri (NRPPa, LPP).
`lib/ru`, `lib/radio`	–	RU taklidi (`ru/dummy`), radyo sürücüleri (`radio/zmq`, `radio/uhd`).
`configs/`	–	Hazır yapılandırma örnekleri (`testmode.yml` dahil).
`docs/`, `lib/*/README.md`	–	Mimari dokümanlar.

Satır sayıları `find ... | xargs cat | wc -l` ile ölçüldü (yalnızca `.cpp` ve `.h`); "–" ölçülmedi.

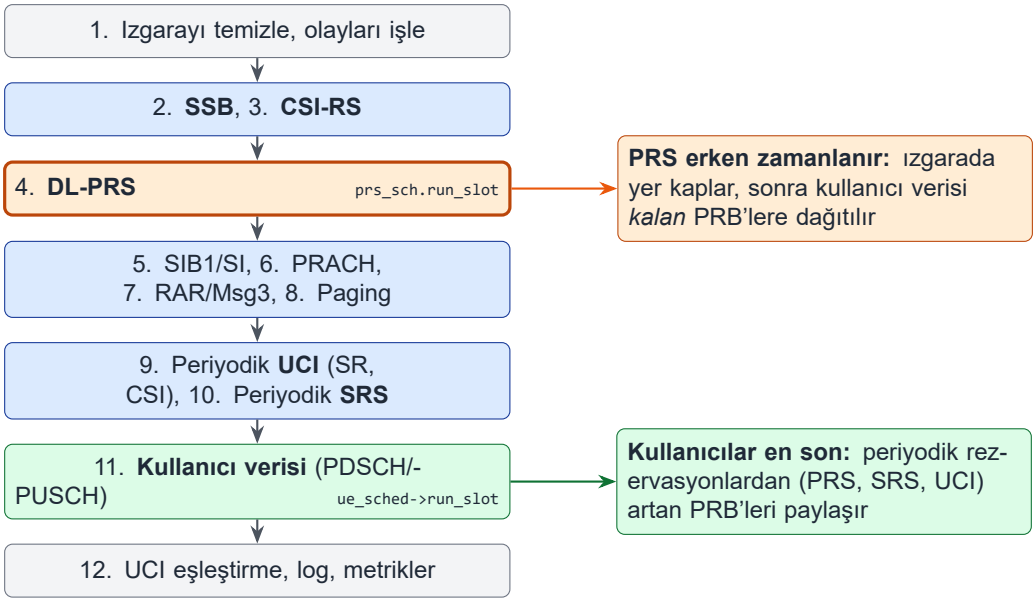
Kodu nereden okumaya başlamalı?

`lib/scheduler/README.md` (scheduler'ın tüm bileşenlerini ve slot akışını anlatır) ve `lib/README.md`. İkisi de iyi yazılmış ve güncel. Koda dalmadan önce bu iki dosyayı okumak, doğru dosyayı bulmak için gereken

zamanı kısaltır.

5.2 Scheduler bir slotta ne yapar?

`cell_scheduler::run_slot()` (`lib/scheduler/cell_scheduler.cpp`) **her slot** bir kez çalışır ve kaynak ızgarasını doldurur. Kodun gerçek sırası (satır yorumlarından):



Şekil 25. OCUDU scheduler'ının slot başına işlem sırası (`cell_scheduler.cpp`). Turuncu adım: **sensing sinyali olarak kullanabileceğimiz PRS**.

Anahtar fikir: *periyodik, önceden bilinen* sinyaller (SSB, CSI-RS, **PRS**, PRACH, UCI, SRS) önce ızgarada yerini alır; kullanıcı verisi *kalan kaynağı* alır. Dokümandaki ifadeyle: "*uci_scheduler ve srs_scheduler, UE tahsisinden önce periyodik fırsatlarını rezerve etmek zorunda olduğu için UE zamanlamadan önce çalışır*". Sensing'in kaynak ihtiyacı da tam bu türdendir: periyodik ve önceden bilinen.

5.3 Büyük bulgu: OCUDU'da PRS zaten var

Sensing için ihtiyaç duyduğumuz *gönderici tarafının* büyük bölümü OCUDU'da hazır:

Dosya	Ne yapıyor?
`lib/scheduler/common_scheduling/prs_scheduler.cpp`	**MAC'te DL-PRS zamanlayıcısı.** PRS kaynak kümelerini periyoda, ofsete, tekrar faktörüne, *muting* desenlerine göre slotlara yerleştirir.
`lib/phy/upper/signal_processors/prs/prs_generator_impl.cpp`	**PHY'de PRS dizisini üretir** (standarttaki dizi, TS 38.211 §7.4.1.7).
`lib/fapi_adaptor/mac/p7/pdu_translators/prs.cpp` ve `.../phy/p7/.../prs.cpp`	MAC'in PRS isteğini PHY'nin anlayacağı biçime çevirir.
`lib/f1ap/cu_cp/procedures/f1ap_positioning_*.cpp`	CU–DU arasında konumlandırma prosedürleri.
`lib/nrppa/`, `lib/lpp/`	Konumlandırma protokolleri.

PRS'in kapladığı kaynağı rezerve eden asıl satır (`prs_scheduler.cpp`, `schedule_slot`):

```
slot_alloc.dl_res_grid.fill(grant_info{pdu.scs, pdu.symbols, pdu.crbs});
```

Yani PRS'in kapladığı *sembol aralığı* ($\times$ PRB aralığı), ızgarada "dolu" olarak işaretlenir. Kullanıcı verisi scheduler'ı bu bölgeye tahsis yapamaz.

5.3.1 PRS yapılandırması (YAML)

PRS, hücre yapılandırmasında `cell_cfg:` altında `prs:` bölümüyle açılır. Bu yapılandırmayı denedik; gNB hücreyi açtı. Örnek (PRS her 4 slotta bir, 20 MHz hücre, 104 PRB):

```
cell_cfg:
  dl_arfcn: 368500
  band: 3
  channel_bandwidth_MHz: 20
  common_scs: 15
  prs:
    resource_sets:
      - comb_size: 2
        periodicity_slots: 4
        slot_offset: 1
        nof_symbols: 12
        bandwidth_prbs: 104
        resources:
          - sequence_id: 1
            re_offset: 0
            slot_offset: 0
            symbol_offset: 2
```

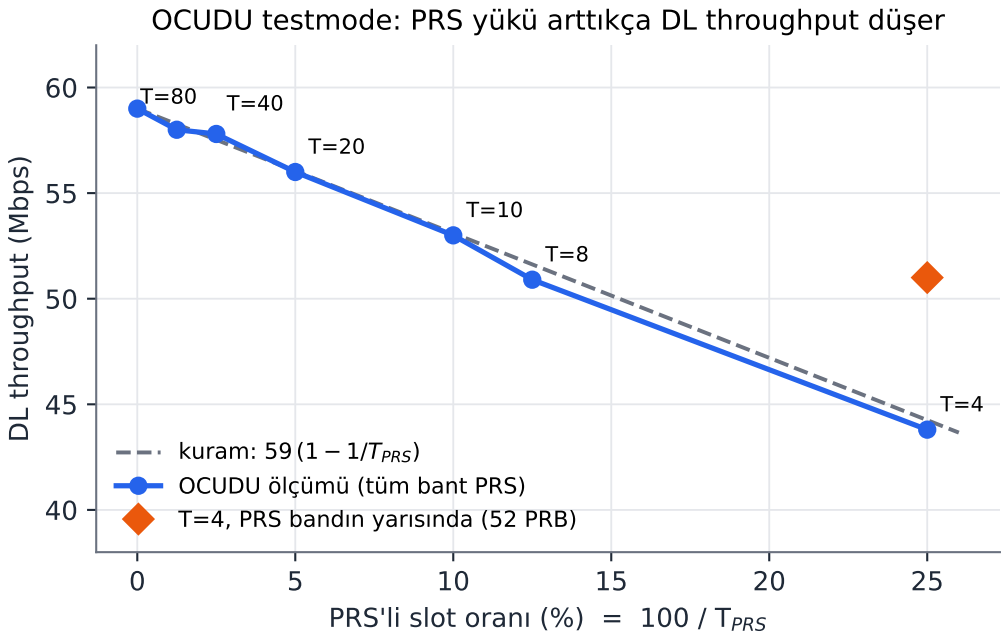
Parametre	Anlamı	Geçerli değerler
`periodicity_slots`	PRS kaç slotta bir tekrarlanır (T_{PRS})	4, 5, 8, 10, 16, 20, 32, 40, 64, 80, 160, …
`slot_offset`	Periyot içindeki başlangıç slotu	$0 \ldots T-1$
`nof_symbols`	Her kaynağın OFDM sembol sayısı	2, 4, 6, 12
`bandwidth_prbs`	PRS'in kapladığı PRB (4'ün katı)	24 …272
`comb_size`	Comb (alt taşıyıcı seyreltme)	2, 4, 6, 12
`repetition_factor`, `time_gap`	Tekrar sayısı ve aralığı	1–32
`power_offset_db`	Güç ofseti	$-60 \ldots 50$

OCUDU yapılandırmayı doğrular

`slot_offset: 0` ve periyot 10 verdiğimizde gNB *başlamadı*: `Detected a collision between the SSB occasion (ssb-Index=0) and the DL-PRS resource ... in slot 0 (period 10 slots).` PRS, SSB ile çakışamaz; `slot_offset: 1` ile çözüldü. Ayrıca PDSCH için `mcs_table: qam64` verilmezse "256QAM MCS table cannot be used for PDSCH with fallback DCI format" hatası alınır. Bunlar *doğrulama* mesajlarıdır, hata değil.

5.3.2 Ölçüm: PRS yükü arttıkça DL throughput ne kadar düşer?

OCUDU'yu radyosuz modda (`testmode`, `ru_dummy`, 20 MHz FDD, 15 kHz SCS, 1 sanal UE, tam tamponlu indirme trafiği) çalıştırıp PRS periyodunu değiştirdik. Her koşu yaklaşık 30 s, tablodaki değerler son 15 satırın ortalamasıdır:



Şekil 26. Gerçek OCUDU ölçümü (WSL2, testmode). Mavi: tüm banda PRS (104/106 PRB). Kesikli çizgi: kuram $59\,(1 - 1/T_{PRS})$. Turuncu: $T = 4$ ama PRS yalnızca 52 PRB'de. Veri: `ogretici/veri/ocudu_prs_sweep.csv`.

Koşul	T_{PRS}	sembol	PRB	**DL (Mbps)**	başarılı blok/s
Sensing yok	–	–	–	59,0	1000
$T = 80$	80	12	104	58,0	987
$T = 40$	40	12	104	57,8	972
$T = 20$	20	12	104	56,0	950
$T = 10$	10	12	104	53,0	900
$T = 8$	8	12	104	50,9	874
$T = 4$	4	12	104	43,8	749
$T = 4$, yarım bant	4	12	52	51,0	1000
$T = 4$, 4 sembol	4	4	104	43,8	748

UL throughput tüm koşullarda ≈77,2 Mbps, yani PRS'ten etkilenmedi (PRS DL sinyalidir).

Dört gözlem ve açık soru

(1) H1 ile uyumlu. Tüm banda PRS koyunca DL throughput, PRS'li slot oranı kadar düşüyor: $T = 4$ (%25 slot) ⇒ 59,0 → 43,8 Mbps (%26 azalma). 1000 slotluk saniyede 749 başarılı blok.

(2) Bant payı önemli. PRS'i bandın yarısına (52 PRB) koyunca, aynı periyotta *slot kaybı olmadı* (1000 blok) ve düşüş yalnızca 59,0 → 51,0 Mbps (%13,6): yaklaşık $0{,}25 \times \frac{52}{106} = \%12{,}3$. Scheduler, PRS'in kaplamadığı PRB'lere veri tahsis etmeye devam ediyor.

(3) UL etkilenmiyor. PRS bir DL sinyali olduğu için UL değişmedi.

(4) Açık soru: sembol sayısı neden etkisiz? 12 sembollük PRS ile 4 sembollük PRS *aynı* kaybı verdi (43,8 Mbps). Kuramda 4 sembol daha az kaynak kaplamalı. Olası açıklama: PDSCH tahsisinin zaman ekseni sabit bir sembol aralığına yayılıyor ve PRS ile çakışan PRB'ler o slotta tümden kullanılamıyor. Bu **doğrulanmadı**, OCUDU'daki PDSCH tahsis koduna (`lib/scheduler/ue_scheduling/`) bakılarak cevaplanmalı. Cevap, "sensing kaynak payı" tanımımızı doğrudan etkiler (TR 38.765'teki Tip 3 kaynak!).

Bu ölçümün sınırları

(i) Her koşul *tek koşu*, 15 saniyelik ortalama; tekrarlayıp hata çubuğu eklemek gerekir. (Betikleri taşıdıktan sonra $T = 20$ koşusunu bir kez daha çalıştırdık ve *aynı* değerleri aldık: 56 Mbps, 950 blok/s. Yani bu ortamda koşular arası oynaklık küçük görünüyor, ama bunu 5 koşuyla kanıtlamak gerekir.) **(ii)** WSL2

gerçek zamanlı değil; log'da `Real-time failure in RU: received late DL request` uyarıları vardı. **(iii)** `testmode` gerçek radyo, gerçek kanal ve gerçek UE içermez: yalnızca MAC/PHY'yi uyarır. Sayıların *mutlak değeri* (59 Mbps) gerçek bir sistem için geçerli değildir; *eğilim* geçerlidir. **(iv)** Bu deney sensing'i değil, yalnızca *PRS'in kaynak maliyetini* ölçer. Alıcı tarafı (yankıyı işleyen) yoktur.

5.4 Slicing: PRB oranıyla kaynak ayırma

OCUDU'da kullanıcı trafiği *dilimlenebilir* (network slicing). Her dilim için yapılandırmada (`du_high_config.h`, `du_high_unit_cell_slice_sched_config`):

- `min_prb_policy_ratio`: dilime garanti edilen en az PRB yüzdesi,
- `max_prb_policy_ratio`: en çok PRB yüzdesi,
- `ded_prb_policy_ratio`: dilime *ayrılmış* (dedicated) PRB yüzdesi,
- `priority`: dilim önceliği.

Ayrıca çalışma anında dilim oranı `rrm_policy_ratio_set` uzaktan komutuyla değiştirilebilir. Scheduler'ın içinde `inter_slice_scheduler` dilimleri sıralar, `intra_slice_scheduler` dilimin içinde dağıtır. Scheduler politikaları: `time_rr` (round-robin) ve `time_qos` (QoS-farkında, PF katsayısı yapılandırılabilir).

Fikir (denenmedi): "sensing" için bir dilim tanımlayıp `ded_prb_policy_ratio` ile kaynak ayırmak, PRS'e alternatif ve *koda dokunmadan* bir sensing-yükü emülasyonu olabilir. Bunun gerçekten iş görüp görmediği bir pilot görevdir (Bölüm 6).

5.5 Nereye ekleme yapabiliriz?

Gerçek bir kod tabanına ekleme yaparken **en az değişiklikle başlayıp gerekirse artırmak** esastır. Dört seviye:

	Seviye	Ne yapılır	Ne öğrenir/elde ederiz	Emek
A	**Yapılandırma (kod yok)**	`prs:` bölümünü ve dilimleri YAML'da ayarlayıp `testmode` ile ölç	PRS yükünün haberleşmeye maliyeti (**yaptık**, Şekil 26)	düşük
B	**Ölçüm/log ekleme**	Scheduler metriklerinden PRS'li slot oranı, kaybolan PRB, Tip 1/2/3 kaynakları çıkarmak	TR 38.765'in "sensing kaynak oranı" metriğini *gerçek kodda* hesaplamak	düşük–orta
C	**Politika ekleme**	Yüke göre PRS periyodunu/bandını uyarlayan basit bir kural (α yük arttıkça küçülür)	H3: *uyarlanır* politikanın sabitten iyi olup olmadığı	orta–yüksek
D	**Alıcı tarafı sensing**	Yankı işleme (range-Doppler), SenF/NsAP arayüzü	Tam ISAC. OCUDU'da yok; bizim kapsamımız dışı	çok yüksek

Önerimiz: **A** ve **B**'yi hedefleyin; **C**'yi "zaman kalırsa" bırakın; **D**'yi yalnızca *tasarım diyagramı* olarak (bkz. Bölüm 3.4) yapın. Bunun nedeni basit: ana sorumuz haberleşme tarafına etkiyi ölçmek, A–B bunu zaten verir.

5.5.1 Kod geliştirme döngüsü (A ve B için)

```
# 1) Kodu degistir (Windows'ta VS Code ile, Remote-WSL uzerinden)
cd ~/ehb415/ocudu && code .

# 2) Sadece gnb'yi yeniden derle (degisen dosyalar icin hizli, ccache sayesinde)
cd ~/ehb415/ocudu/build && make -j4 gnb
```

```
# 3) Calistir (yapilandirma dosyalarini birlestirebilirsin: -c a.yml -c b.yml)
./apps/gnb/gnb -c /mnt/c/Users/<kullanici>/Desktop/EHB415/kurulum/ocudu_testmode.yml
# gNB acildiktan sonra konsola "t" yazip Enter: metrik tablosu gorunur (bu makinede boyle gozlendi)
```

Gerçek bir geliştirici gibi çalış

Her değişiklikten sonra `git diff` ile ne değiştiğine bak ve *küçük adımlarla* ilerle: bir ölçüm satırı ekle, derle, çalıştır, sonucu gör. Büyük değişiklikleri tek seferde yapmak, OCUDU gibi yüz binlerce satırlık bir kod tabanında hatayı bulmayı imkânsız hale getirir. Ayrıca kendi değişikliklerini *yama (patch) dosyası* olarak saklayın; OCUDU deposunun kendisini projemizin reposuna kopyalamıyoruz.

6 Pilot görevler ve görev dağılımı

6.1 Pilot hafta: herkes aynı üç mini deneyi yapar

Amaç: Herkes, uzmanlaşmadan *önce*, üç aracın her birinde bir sonuç üretsin. Böylece "OCUDU'cu" ns-3'ün ne yaptığını, "Sionna'cı" scheduler'ın etkisini görmüş olur ve toplantılarda aynı dili konuşuruz. Her pilot yarım gün sürer. Kurulumlar `kurulum/` klasöründeki dokümanlarla yapılmış olmalı.

Pilot	Araç	Ne yapılır	Çıktı
Pilot-1	OCUDU	PRS yükünü değiştirip DL throughput'u ölç	Tablo + 1 paragraf
Pilot-2	ns-3 / 5G-LENA	UE sayısına göre throughput ve gecikmeyi ölç	Tablo + 1 grafik
Pilot-3	Sionna (radar)	Range-Doppler haritası, çözünürlük ve boşluk deneyi	1 grafik + 1 paragraf

Her pilotun sonunda bir **soru kartı** var: kendi cümlelerinizle cevaplayın (her biri 2–3 cümle). Ek B'te *beklenen cevaplar* yönetici için verilmiştir; cevapları yazmadan önce bakmayın, öğrenmenin yolu o.

6.1.1 Pilot-1: OCUDU'da PRS maliyetini ölç

Pilot-1: adımlar

1. Ubuntu'da: `cd ~/ehb415/ocudu/build/apps/gnb`
2. Temel koşu (PRS yok): `./gnb -c /mnt/c/Users/<kullanici>/Desktop/EHB415/kurulum/ocudu_testmode.yml`. gNB açılınca `t` yazıp Enter; tablo gelir. DL `brate` sütununu not et (59 M civarı). `Ctrl+C` ile kapat.
3. Bir kopya yapılandırma dosyası hazırla, `cell_cfg:` altına `pdsch:`'den önce Bölüm 5.3'deki `prs:` bloğunu ekle. Hazır örnek: `ogretici/veri/ocudu_p4_config.yml` ($T = 4$).
4. Aynı şekilde çalıştır; `brate` ve `ok` sütunlarını not et. T değerini 8, 20, 40 yap, her seferinde tekrarla.
5. `bandwidth_prbs: 52` ile $T = 4$'ü dene.

Kısayol: adım 3–5'i otomatikleştiren betik: `bash ogretici/betikler/ocudu_prs_kos.sh p4 4 12 104` (etiket, periyot, sembol, PRB). Önce elle yapın, sonra betiği kullanın. **Çıktı:** T ve `bandwidth_prbs`'e göre DL throughput tablosu. Bölüm 5.3'deki tabloyla karşılaştır.

Soru kartı: Pilot-1

(1) PRS her 4 slotta bir olunca DL throughput neden yaklaşık %25 düştü? **(2)** PRS'i bandın yarısına koyunca neden yalnızca yaklaşık %13 düştü? **(3)** Bu deney TR 38.765'teki Tip 1, 2 ve 3 kaynakların hangisine karşılık gelir?

6.1.2 Pilot-2: ns-3'te UE sayısını artır

Pilot-2: adımlar

1. Ubuntu'da: `cd ~/ehb415/ns-3-dev`
2. `./ns3 run "cttc-nr-demo --ueNumPergNb=1"`, sonra 2, 5, 10, 20 için tekrarla.
3. Her koşuda çıktıdaki akış sayısını, her akışın `Throughput` ve `Mean delay` değerini ve son satırdaki `Mean flow throughput/Mean flow delay`'i not et.
4. Toplam throughput'u (akışların toplamı) ve ortalama gecikmeyi UE sayısına karşı tabloya yaz; Excel veya Python ile grafiğe çevir.

Çıktı: Bölüm 4.2'teki gibi bir tablo ve grafik.

Soru kartı: Pilot-2

(1) Gecikme neden 0,28 ms'den yaklaşık 124 ms'ye çıktı? **(2)** Toplam throughput neden UE sayısıyla doğrusal artmadı? **(3)** Sensing yükünü bu senaryoya koymak için Bölüm 4.2.2'deki (a), (b), (c) yollarından hangisini önerirsin, neden?

6.1.3 Pilot-3: Radar ve boşluk deneyi

Pilot-3: adımlar

1. Windows PowerShell'de: `C:\Users\<kullanici>\venvs\ehb415\Scripts\python.exe kurulum\sionna_dogrulama.py`. Çıktıdaki gerçek ve tahmin edilen menzil/hızı not et.
2. `ogretici\betikler\make_figs.py` içindeki `fig_gap()` fonksiyonunu çalıştır (`pip install pandas matplotlib` gerekebilir).
3. Betikte hedef mesafesini ve hızını değiştir, haritada tepenin de kaydığını gör.
4. Sembol sayısını $M = 128$'den 32'ye düşür; çözünürlüğün nasıl değiştiğini formülle karşılaştır.

Çıktı: Bölüm 4.1'deki gibi bir grafik ve "neden?" paragrafı.

Soru kartı: Pilot-3

(1) $M = 128$ yerine 32 sembol kullanınca hız çözünürlüğü kaç kat değişir? **(2)** Boşluklar neden yan lobları yükseltir? **(3)** Haberleşme scheduler'ı sensing için *neyi garanti etmeli*?

6.2 Dört kişilik görev dağılımı

Pilotlardan sonra herkes bir **rol** üstlenir. Her rolün somut bir "Faz-1 çıktısı" var (çıktısı olmayan görev vermiyoruz). Dördüncü rol, proje yöneticisinin entegrasyon ve standart işi.

Rol	Alan	Faz-1 çıktıları (somut)	Önce okuma
R1	**Standart, metrik, entegrasyon**	(a) `docs/metrics.md`: TR 38.765 §4.1 ve §6.2'den metrik sözlüğü (formüllerle). (b) Dickerson modelini (α, $T(\eta)$) bir tabloya/betiğe dök. (c) Üç aracın sonuçlarını *aynı* α *eksenine* getiren ortak grafik şablonu.	TR 38.765 §4, 6.2, 9; Lin [2]; Dickerson [6]
R2	**Sionna**	(a) Sensing performansı eğrisi: bant (W) ve sembol sayısına (M) göre menzil/hız hatası ve tespit olasılığı. (b) Boşluk deneyini gerçekçileştir (SNR, çok hedef). (c) Sionna RT öğreticilerinden 3'ünü çalıştır.	Wei [9], Khosroshahi [7], NIST [8]; Sionna RT [16]
R3	**ns-3 / 5G-LENA**	(a) Sensing yükünü (a) ve (b) yollarıyla ekle (Bölüm 4.2.2). (b) α'ya göre throughput/gecikme/PLR taraması, CSV + grafik. (c) 5G-LENA scheduler sınıflarının tablosu.	SioLENA [12]; Koutlia [13]; Capozzi [17]

R4	**OCUDU**	(a) Bölüm 5.3'deki ölçümü *tekrarla ve hata çubuğu ekle* (örn. 5 koşu). (b) "Sembol sayısı neden etkisiz?" sorusunu kodu okuyarak cevapla. (c) Slicing ile (`ded_prb_policy_ratio`) sensing emülasyonunu dene. (d) Kod haritası ve "sensing nereye takılır" diyagramı.	Lindenschmitt [10]; Gurung [11]; OCUDU `lib/scheduler/README.md`

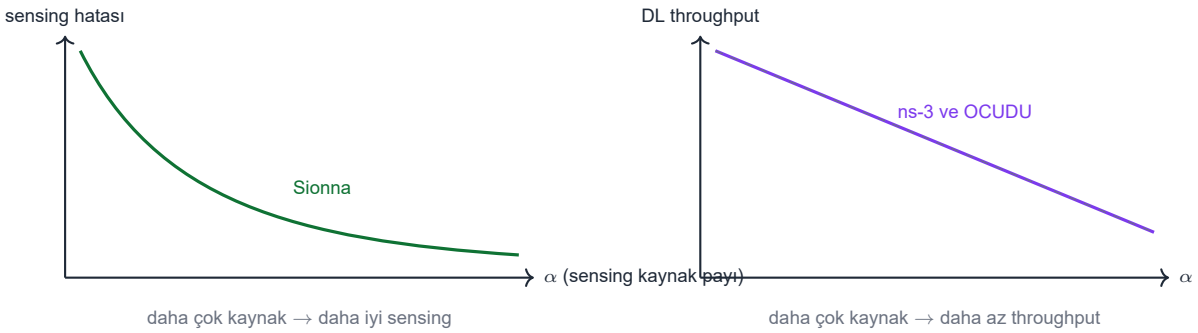
Ortak çalışma ("ikinci göz"): her çıktı bir başkası tarafından *yeniden üretilmeden* "bitti" sayılmaz. Öneri eşleşmeleri: R2 ↔ R3 (kanal/ağ köprüsü, SioLENA), R3 ↔ R4 (ns-3 sonuçlarıyla OCUDU ölçümünü karşılaştırma), R1 ↔ herkes (metrik tutarlılığı).

Rolleri neden böyle böldük?

Üç araç birbirinden *bağımsız* çalışabilir, ama sonuçları **aynı eksende** buluşmalı. Bunu garanti eden rol R1'dir: herkes sonucunu α cinsinden ve R1'in metrik sözlüğündeki tanımlarla raporlar. Aksi halde üç farklı "overhead" tanımıyla üç uyumsuz grafik çıkar.

6.3 Hedef grafik: hepimizin üzerinde çalıştığı tek şekil

Projenin "kahramanı" şekil şu şemadaki gibi olacak (**bu bir şematiktir, veri değildir**):



Şekil 27. Şematik (veri değil): iki ters eğri aynı α ekseninde. Proje, bu iki eğrinin gerçek hâlini üretmek ve "çalışma noktası" (hem sensing yeterli hem haberleşme kabul edilebilir) bölgesini bulmaktır.

6.4 Haftalık plan önerisi

Kesin teslim tarihlerini bilmediğimden aşağıdaki **12 haftalık bir öneridir**; ara rapor/final tarihine göre ölçeklenmelidir.

Hafta	Hedef
1	Kurulumlar bitti. **Pilot-1, 2, 3** (herkes). Soru kartları cevaplandı.
2	Okuma: herkes Bölüm 1–3 + kendi rolünün makaleleri. **Karar toplantısı:** sensing yükünü nasıl temsil ediyoruz (Bölüm 4.2.2 a/b/c), hangi metrikler.
3–4	Rol çıktıları (Faz-1). Haftalık 30 dakikalık "gördüğümü anlat" toplantısı.
5	**İlk entegrasyon:** üç aracın sonucu aynı α ekseninde, taslak ortak grafik.
6	**Ara rapor.** Boşluklar ve riskler listelenir.
7–9	Genişletme: OCUDU seviye B (metrik/log), Sionna RT ile gerçekçi kanal, ns-3'te (c) yolu veya SioLENA (zaman kalırsa).
10	Tekrar koşular, hata çubukları, sınırlılıklar bölümü.
11	Rapor yazımı.
12	Gözden geçirme, tekrar üretilebilirlik kontrolü (başkası komutlarla aynı sonucu alıyor mu?), teslim.

6.5 Riskler ve nasıl yönetiriz

Risk	Etkisi	Önlem
Ölçümler tek koşu, WSL2 gerçek zamanlı değil	Sayılar oynak olabilir	R4: 5 koşu + hata çubuğu
"Sembol sayısı etkisiz" bulgusunun nedeni belirsiz	Tip 3 kaynak tanımımız yanlış olabilir	R4: PDSCH tahsis kodunu oku (Bölüm 5.3)
5G-LENA'da sensing modeli yok	Deney tasarımı belirsiz	Karar toplantısı (hafta 2), (a)/(b) yolu
SioLENA ayrı ns-3 ağacı, eski Sionna	Entegrasyon zor	İkinci aşama; ana sonuca bağımlı yapma
Rapor yazılmadan zaman biter	Teslim riski	Hafta 5'te taslak grafik, hafta 11 yazım

7 Makale okuma rehberi

7.1 Google Scholar'a ne yazmalı?

Scholar, arama kutusundaki **tırnak içindeki ifadeleri aynen** arar ve çok kelimeli sorgularda çok geniş sonuç döndürür. Bu yüzden *tırnaklı anahtar ifadeler* ve yanlarında 1–2 serbest kelime kullanın. Aşağıdaki sorgular, bu projenin her alt konusunu kapsayacak şekilde tasarlandı. **Sorgulardaki sonuçları ben kontrol etmedim**; sonuç sayısı ve sırası değişir. Bulduklarınızı Bölüm 7.3'teki listeyle karşılaştırın.

	Sorgu (kopyala-yapıştır)	Ne ararsın / ne işine yarar
1	`"integrated sensing and communication" "5G NR" "positioning reference signal"`	PRS ile sensing yapan çalışmalar; sinyal tasarımı ve sonuçlar.
2	`"ISAC" "resource allocation" "5G NR" "sensing overhead"`	**Ana sorumuz:** sensing kaynak payı ve throughput kaybı.
3	`"ISAC" "3GPP" "Release 19" OR "Release 20" sensing "channel model"`	Standart durumu, kanal modeli.
4	`"gNB" "monostatic sensing" UAV detection "5G NR"`	TR 38.765'in senaryosuyla aynı çalışmalar.
5	`"ISAC" "O-RAN" OR "OpenRAN" xApp sensing srsRAN OR OpenAirInterface`	Açık RAN'a sensing entegrasyon mimarileri (OCUDU tarafı).
6	`"Sionna" "ray tracing" sensing OR radar OFDM`	Sionna ile sensing/ISAC uygulamaları, hazır kod.
7	`"ns-3" "5G-LENA" sensing OR ISAC`	ns-3'te sensing modelleme (az çıkması beklenir; boşluğu gösterir).
8	`"OFDM radar" "range-Doppler" "mobile network" tutorial`	Radyo sinyal işleme temelleri, öğretici nitelikli.
9	`"self-interference" "monostatic" "base station" sensing isolation`	Öz-girişim, neden zor, hangi izolasyon yeter.
10	`"ghost targets" PRS comb sensing`	PRS tabanlı sensing'in bilinen sorunu ve çözümleri.
11	`"communication-sensing tradeoff" OR "sensing-communication tradeoff" ISAC "Cramer-Rao"`	Teorik ödünleşim (R1 için, matematik yoğun).
12	`"sensing-assisted" scheduling OR "beam management" 5G ISAC`	Tersine yön: sensing'in haberleşmeye *katkısı* (rapora zenginlik).

7.1.1 Scholar'ı verimli kullanmak

- Soldaki menüden **"Since 2024"** (veya "Since 2025") seçin: alan hızlı değişiyor, 2022 öncesi çalışmalar çoğu zaman eski.
- **"Cited by"** bağlantısı: bir makaleyi *atıf yapan yeni makaleler* üzerinden ileriye doğru izleyin (en değerli ipucu).

- **"Related articles"** ve **"Versions"**: arXiv sürümünü bulun, çoğu zaman ücretsizdir.
- **"Cite"** → BibTeX: rapor kaynakçası için.
- Bir "survey" veya "tutorial" bulursanız *önce onu* okuyun: yüzlerce makaleyi sizin için özetler (örn. Liu vd. [5]).

7.2 Bir makalede ne aranır? (7 soru)

Makaleyi baştan sona okumayın. Önce **özet, giriş sonu (katkılar) ve sonuç**, sonra şu yedi soruya cevap arayın:

1. **Ne ölçüyorlar?** (Hangi metrik: P_{md}, hata, throughput?) Bizim metriklerimizle aynı mı?
2. **Sistem modeli ve parametre tablosu:** bant, SCS, taşıyıcı, anten, hedef. Bizimkiyle karşılaştırılabilir mi?
3. **Sensing sinyali ne?** (PRS, SRS, DMRS, özel dizi?) Haberleşmeyle nasıl paylaşıyor?
4. **Kaynak paylaşımı nasıl?** (Zaman, frekans, güç, uzay; sabit mi uyarlanır mı?)
5. **Hangi araç/simülatör?** (MATLAB, Sionna, ns-3, OAI, srsRAN/OCUDU?) *Kodu paylaşıyorlar mı?*
6. **Sınırlamalar ve "future work"** bölümü: bizim için *fırsat* (onların yapmadığı şeyi biz yapıyoruz).
7. **Bizim projeye tek cümlelik katkısı:** "X makalesi, Y için Z'yi veriyor." Bu cümleyi yazamıyorsanız makaleyi bırakın.

Her makale için 5 dakikalık not

Her okuduğunuz makale için ortak bir tabloya **tek satır** ekleyin: `Makale | Sinyal | Araç | Kod var mı | Bizim için katkı (1 cümle) | Güven (düşük/orta/yüksek)`. Dönem sonunda ilgili çalışmalar bölümünün neredeyse hazır olacak.

7.3 Okuma listesi (indirildi, doğrulandı)

Aşağıdaki makaleler `makaleler/` klasöründe PDF olarak durur; başlık, yazar ve tarihleri arXiv/3GPP'den doğrulandı. Ayrıntılı "şurada ne var" notları `makaleler/OKUMA_LISTESI.md` dosyasında.

Kim	Makale	Önce oku	Neden
Herkes	Lin, *ISAC in 3GPP: 5G-Adv. to 6G* [2]	Tamamı (8 s.)	Standart durumunun güncel ve kısa özeti.
Herkes	3GPP TR 38.765	§4, 6.2, 9	Metrikler, kaynak oranı, kararlar.
R1, R3	Dickerson vd. [6]	§III.B, IV.A	Kaynak payı modeli ve throughput formülü.
R2	Khosroshahi vd. [7]	Özet, §resource allocation	PRS+PDSCH Pareto optimali.
R2	Blandino vd. (NIST) [8]	Özet, sonuçlar	İHA tespiti sonuçları; kod (MATLAB) var.
R2	Wei vd. [9]	CRLB bölümü	Kuramsal sınır; sonuçlarımızı kıyaslamak için.
R3	Ropitault vd. (SioLENA) [12]	§III.B, III.C	Sionna RT → 5G-LENA kanal köprüsü.
R3	Capozzi vd. [17]	§IV.B–C, V	Scheduler aileleri ve metrik tanımları.
R4	Lindenschmitt vd. [10]	§3.3, 4.4, 5	Sensing işlemenin RAN'daki yeri ve açık sorular.
R4	Gurung vd. [11]	§III–IV	OAI + RIC + Sionna RT ile ISAC test düzeneği.
İsteğe bağlı	Liu vd. [5]	§III, IV.A	Alanın temel tutorial'ı; trade-off teorisi.

A Sözlük

Terim	Anlamı
gNB	5G baz istasyonu.
UE	Kullanıcı cihazı (telefon).
NR	5G'nin radyo arayüzü (New Radio).
FR1	6 GHz altı frekans aralığı (bizim senaryolar: 4–4,9 GHz, band n3 testlerinde 1,8 GHz).
OFDM	Veriyi çok sayıda dar alt taşıyıcıya yayan modülasyon. 5G'nin ve OFDM radarın temeli.
SCS	Alt taşıyıcı aralığı (15, 30, 60, 120 kHz). Slot süresini belirler.
Slot / sembol	Zaman birimi: bir slot 14 OFDM sembolüdür.
PRB	12 alt taşıyıcılık frekans birimi (kaynak bloğu).
RE	Kaynak elemanı: 1 sembol $\times$ 1 alt taşıyıcı.
PDSCH / PDCCH	DL veri / DL kontrol kanalı.
SSB	Senkronizasyon sinyali bloğu (hücreyi bulmak için).
PRS	Konumlandırma referans sinyali (Rel-16); sensing için aday.
SRS / DMRS	UL ses/kanal ölçüm sinyali / veri çözme referans sinyali.
Scheduler	Slot başına kaynakları kullanıcılara dağıtan MAC bileşeni.
RR / PF	Round-Robin / Proportional Fair scheduler politikaları.
MAC / PHY / RLC / PDCP	Protokol katmanları (MAC: zamanlama; PHY: fiziksel işlem).
CU / DU / RU	Merkezi / dağıtık / radyo birimi (gNB'nin bölünmüş yapısı).
O-RAN, RIC	Açık RAN mimarisi; RAN Akıllı Denetleyicisi (dışarıdan kontrol/ölçüm).
FAPI	MAC ile PHY arasındaki arayüz.
F1, E1, NG	CU–DU, CU-CP–CU-UP ve gNB–çekirdek ağ arayüzleri.
ISAC	Entegre sensing ve haberleşme.
Monostatic / bistatic	Gönderici ve alıcının aynı / ayrı yerde olduğu radar düzeni.
Range-Doppler haritası	Menzil ve hızı gösteren iki boyutlu radar çıktısı.
RCS	Radar kesit alanı: hedefin ne kadar yansıttığı.
Clutter	Hedef olmayan yansımalar (bina, ağaç).
Hayalet hedef (ghost)	Sinyal yapısından doğan sahte tepe noktası.
Öz-girişim	Vericinin kendi alıcısına sızan sinyali (monostatic sensing'in ana sorunu).
SNR / SINR	Sinyal-gürültü / sinyal-girişim-gürültü oranı.
P_{md}, P_{f1}, P_{f2}	Kaçırma olasılığı, yanlış alarm tip 1 ve tip 2 (TR 38.765).
CRLB	Cramér–Rao sınırı: bir kestirimin ulaşabileceği teorik en iyi doğruluk.
α (sensing kaynak payı)	Sensing'e giden kaynak oranı; projenin ortak ekseni.
Tip 1/2/3 kaynak	TR 38.765'te sensing kaynak oranı tanımı (Bölüm 3).
SenF / NsAP	Çekirdekteki sensing işlevi / gNB–SenF protokolü (Rel-20).
TR / TS	3GPP Teknik Rapor / Teknik Şartname.
Release (Rel-N)	3GPP standart sürümü.
WSL2	Windows içinde çalışan Linux (Ubuntu) ortamı.
ZMQ / testmode / ru_dummy	OCUDU'da radyosuz çalıştırma yolları (sanal radyo, test UE, sahte RU).
Pareto	Bir hedefi iyileştirirken diğerini kötüleştirmeden daha iyi bir nokta kalmaması.

B Soru kartlarının beklenen cevapları (yönetici için)

Önce kendiniz cevaplayın

Pilot yapan kişiler cevapları yazmadan bu sayfaya bakmamalı. Aşağıdakiler *referans cevaplardır*, tek doğru ifade değil.

Pilot-1 (OCUDU)

1. PRS tüm bandın 104/106 PRB'sini kapladığı için, PRS'li slotta kullanıcı verisine neredeyse PRB kalmaz. $T = 4$'te her 4 slottan biri bu durumdadır, yani kapasitenin yaklaşık $1/4$'ü gider (ölçülen: 59,0 $\rightarrow$ 43,8 Mbps, %26).
2. PRS 52 PRB'de iken scheduler kalan 54 PRB'ye veri vermeye devam eder. Kaybolan kaynak oranı yaklaşık $\frac{1}{4} \times \frac{52}{106} \approx$ %12,3; ölçülen %13,6.
3. PRS'in kapladığı sembol $\times$ PRB **Tip 1**'dir; PRS veri taşımadığı için **Tip 2** yoktur. 4 ve 12 sembollü PRS'in aynı kaybı vermesi, PRS dışındaki sembollerin de kullanılamadığına işaret eder; bu **Tip 3** benzeri bir kayıptır. *Nedeni doğrulanmadı* (R4'ün görevi).

Pilot-2 (ns-3)

1. Sunulan trafik (UE başına yaklaşık 10 veya 102 Mbps) hücre kapasitesini aştıkça kuyruklar uzar. Ölçülen gecikme büyük ölçüde *kuyruk bekleme* gecikmesidir.
2. Radyo kapasitesi sonludur; yük kapasiteyi aşınca yeni UE'ler toplamı artırmaz, yalnızca kapasiteyi bölüşür. Bu yüzden eğri doygunluğa yaklaşır.
3. Önerilen: (b), sensing'i yüksek öncelikli periyodik trafik olarak eklemek: (a)'dan daha gerçekçi (zamansal yapı), (c)'den çok daha ucuz. (c), zaman kalırsa ve OCUDU ile tutarlılık için.

Pilot-3 (Sionna/radar)

1. $\Delta v \propto 1/M$ olduğundan 128'den 32'ye inince hız çözünürlüğü 4 kat kabalaşır.
2. Eksik semboller, zaman ekseninde düzensiz örnekleme demektir; FFT'de bu, enerjiyi hedeften yan loblara dağıtır (spektral sızıntı).
3. Düzenli ve boşluksuz sensing sembolleri: sensing sinyalinin periyodik ve zaman ekseninde kesintisiz olması, haberleşme scheduler'ı tarafından *öncelikli rezerve edilmesi* (OCUDU'daki PRS/SRS gibi).

C Komut kartı

İş	Komut
Ubuntu'ya gir (PowerShell)	`wsl -d Ubuntu-24.04` (çıkış: `exit`)
ns-3 klasörü	`cd ~/ehb415/ns-3-dev`
ns-3 örnek	`./ns3 run "cttc-nr-demo --ueNumPergNb=5"`
ns-3 parametreler	`./ns3 run "cttc-nr-demo --PrintHelp"`
OCUDU çalıştır	`cd ~/ehb415/ocudu/build/apps/gnb && ./gnb -c <yapilandirma.yml>`
OCUDU derle	`cd ~/ehb415/ocudu/build && make -j4 gnb`
Sionna doğrulama	`...\venvs\ehb415\Scripts\python.exe kurulum\sionna_dogrulama.py`
OCUDU PRS taraması	`bash ogretici/betikler/ocudu_prs_sweep.sh` (yaklaşık 6 dk; çıktı `/tmp/prs_sweep.csv`)
ns-3 UE taraması	`bash ogretici/betikler/ns3_ue_sweep.sh` (yaklaşık 2 dk; çıktı `/tmp/ns3_sweep.csv`)
Şekilleri yeniden üret	`python ogretici\betikler\make_figs.py` (numpy, matplotlib, pandas)

D Ölçüm yöntemi ve sınırlar

- **OCUDU:** `ogretici/veri/ocudu_prs_sweep.csv` (üreten betikler: `ocudu_prs_kos.sh`, `ocudu_prs_sweep.sh`). Her koşu: `gnb` başlatıldı, 8 s sonra `t` gönderildi, 22 s sonra çıkıldı; son 15 satırın ortalaması alındı. Tek koşu. WSL2, gerçek zamanlı değil. Yapılandırma: `ogretici/veri/ocudu_p4_config.yml`.

- **ns-3:** `ogretici/veri/ns3_ue_sweep.csv`. 1 s simülasyon, varsayılan tohum, tek koşu. Toplam throughput, akış throughput'larının toplamıdır.
- **Radar simülasyonları:** `betikler/make_figs.py`. Tek hedef/iki hedef, ideal kanal, toplamsal beyaz gürültü, sabit SNR; boşluk deneyi 12 tohum ortalaması.
- **D1 modeli:** makalenin denklemleri, makalenin parametreleriyle; iki sayı makaleyle birebir doğrulandı (%1,66 ve %13,29).

Kaynakça

[1] 3GPP, *TR 38.765: Study on Integrated Sensing and Communication (ISAC) for NR*, V20.0.0, Haziran 2026.

[2] X. Lin, "Integrated Sensing and Communication in 3GPP: Evolution from 5G-Advanced to 6G," arXiv:2608.11606, 2026.

[3] X. Lin, "A Tale of Two Mobile Generations: 5G-Advanced and 6G in 3GPP Release 20," arXiv:2506.11828, 2025.

[4] Y. Liu, Y. Zhang, J. Zhang *et al.*, "A Comprehensive Survey of 3GPP Release 19 ISAC Channel Modeling," arXiv:2512.03506, 2025. *(Özet düzeyinde okundu.)*

[5] F. Liu, Y. Cui, C. Masouros, J. Xu, T. X. Han, Y. C. Eldar, S. Buzzi, "Integrated Sensing and Communications: Toward Dual-Functional Wireless Networks for 6G and Beyond," *IEEE JSAC*, 2022 (arXiv:2108.07165).

[6] C. Dickerson, W. Khawaja, I. Guvenc, "Adaptive 5G Resource Allocation for Multistatic ISAC-Based UAV Detection and Tracking," arXiv:2606.21677, 2026.

[7] K. Khosroshahi, P. Sehier, S. Mekki, "Leveraging PRS and PDSCH for Integrated Sensing and Communication Systems," arXiv:2408.00667, 2024.

[8] S. Blandino, N. Golmie, A. Sahoo *et al.*, "Detecting Airborne Objects with 5G NR Radars," arXiv:2505.24763, 2025. Kod: `github.com/usnistgov/5GNRad` (MATLAB).

[9] Z. Wei, Y. Wang, L. Ma *et al.*, "5G PRS-Based Sensing: A Sensing Reference Signal Approach for Joint Sensing and Communication System," arXiv:2211.11488, 2022.

[10] D. Lindenschmitt, T. Jung, P. K. Kakani, T. Reissland, N. Franchi, H. D. Schotten, "Analysis of an Architecture for Integrated Sensing and Communication in 5G OpenRAN," arXiv:2509.16917, 2025.

[11] A. K. Gurung, S. K. Sathananthan, "Multi-UAV Tracking Evaluation Using 5G Uplink Signals on an O-RAN ISAC Simulation Testbed," arXiv:2608.10784, 2026.

[12] T. Ropitault, M. Bordin, P. Testolina, M. Polese, P. Johari, N. Golmie, T. Melodia, "Enabling Site-Specific Cellular Network Simulation Through Ray-Tracing-Driven ns-3," arXiv:2508.04004, 2025. Kod: `github.com/usnistgov/siolena`.

[13] K. Koutlia, B. Bojovic, Z. Ali, S. Lagen, "Calibration of the 5G-LENA System Level Simulator in 3GPP reference scenarios," arXiv:2205.03278, 2022.

[14] CTTC, *5G-LENA: the 5G NR module for the ns-3 simulator*, `5g-lena.cttc.es`; kod: `gitlab.com/cttc-lena/nr`.

[15] J. Hoydis, S. Cammerer, F. Ait Aoudia *et al.*, "Sionna: An Open-Source Library for Next-Generation Physical Layer Research," arXiv:2203.11854, 2022.

[16] F. Ait Aoudia, J. Hoydis, M. Nimier-David *et al.*, "Sionna RT: Technical Report," arXiv:2504.21719, 2025.

[17] F. Capozzi, G. Piro, L. A. Grieco, G. Boggia, P. Camarda, "Downlink Packet Scheduling in LTE Cellular Networks: Key Design Issues and a Survey," *IEEE Commun. Surveys & Tutorials*, vol. 15, no. 2, 2013.

[18] OCUDU Project, `gitlab.com/ocudu/ocudu`, `release_26_10`; dokümantasyon: `docs.ocudu.org`.